



Hochschule
Bonn-Rhein-Sieg
University of Applied Sciences



Master's Thesis

Object Detection in Dense Volume Data

Ramit Sharma

Submitted to Hochschule Bonn-Rhein-Sieg,
Department of Computer Science
in partial fulfillment of the requirements for the degree
of Master of Science in Autonomous Systems

Supervised by

Prof. Dr. Paul G. Plöger

Prof. Dr. Ernst Kruijff

September 2021

I, the undersigned below, declare that this work has not previously been submitted to this or any other university and that it is, unless otherwise stated, entirely my own work.

Date

Ramit Sharma

Abstract

This project focuses on object detection in dense volume data. There are several types of dense volume data, namely Computed Tomography (CT) scan, Positron Emission Tomography (PET), Magnetic Resonance Imaging (MRI). This work focuses on CT scans. CT scans are not limited to the medical domain; they are also used in industries. CT scans are used in airport baggage screening, assembly lines, and the object detection systems in these places should be able to detect objects fast. One of the ways to address the issue of computational complexity and make the object detection systems fast is to use low-resolution images. Low-resolution CT scanning is fast. The entire process of scanning and detection can be made faster by using low-resolution images. Even in the medical domain, to reduce the radiation dose, the exposure time of the patient should be reduced. The exposure time of patients could be reduced by allowing low-resolution CT scans. Hence it is essential to find out which object detection model has better accuracy as well as speed at low-resolution CT scans. However, the existing approaches did not provide details about how the model would perform when the resolution of CT scans is varied. Hence in this project, the goal is to analyze the impact of varying resolution of CT scans on both the speed and accuracy of the model. Three object detection models, namely RetinaNet, YOLOv3, and YOLOv5, were trained at various resolutions. Among the three models, it was found that YOLOv5 has the best mAP and f1 score at multiple resolutions on the DeepLesion dataset. RetinaNet model has the least inference time on the DeepLesion dataset. From the experiments, it could be asserted that sacrificing mean average precision (mAP) to improve inference time by reducing resolution is feasible.

Acknowledgements

I would like to extend my respect and gratitude to Prof. Dr. Paul G. Plöger for his constant support and guidance during this thesis. I am also grateful to him for his encouragement and support during the entire course of MAS. Many thanks to Prof. Dr. Ernst Kruijff for being my second supervisor. I am grateful to M.Sc. Laura Anger, my supervisor from Fraunhofer IAIS, for her guidance, encouragement, and support throughout this project. I would also like to extend my gratitude to M.Sc. Ha Duc Bach from Fraunhofer IAIS for his insight and motivation throughout this project. I would also like to extend my deep and earnest gratitude to my advisor M.Sc. Santosh Thoduka for his support and encouragement in this project. I am also thankful to him for reviewing this report. He has helped me throughout this course of MAS, and I am grateful to him. I would also like to thank my friends Lakshadeep Naik and Pradheep Krishna for their help and encouragement and for reviewing my report. I am so lucky to have them as my friends. At last, I would like to thank my parents. I can't explain my gratitude to them in words.

Contents

1	Introduction	1
1.1	Problem statement	2
2	Theoretical Background	3
2.1	Datasets	3
2.1.1	Sparse 3D point cloud	4
2.1.2	Dense volume data	4
2.2	Resolution of image	5
2.3	Anchor Parameters	6
2.4	Architectures of CNN	10
2.4.1	Path Aggregation Network (PANet) [48]	10
2.4.2	Cross Stage Partial Network (CSPNet) [82]	11
2.5	Evaluation metrics	11
2.5.1	Intersection over Union (IoU)	12
2.5.2	Precision	13
2.5.3	Recall	13
2.5.4	F1 score	13
2.5.5	Precision x Recall curve	14
2.5.6	Average Precision(AP)	15
2.5.7	Mean Average Precision (mAP)	17
3	Related work	21
3.1	CT scan Datasets	21
3.1.1	DeepLesion dataset [89]	22
3.2	Object detection overview	24
3.2.1	One stage approach	25

3.2.2	Two-stage approach	31
3.2.3	Summary	39
3.2.4	Discussion	42
4	Methodology	43
4.1	Data Processing	43
4.2	Anchor optimization techniques	43
4.2.1	Differential Evolution algorithm [73]	44
4.3	RetinaNet	45
4.4	YOLO	47
4.4.1	You Only Look Once	47
4.4.2	YOLO9000	51
4.4.3	Yolov3	53
4.4.4	YOLOv5	53
5	Evaluation	55
5.1	Comparison of RetinaNet model with ResNet-50 and ResNet-101 backbone	56
5.2	YOLOv5 models	58
5.3	Initialization with random and pretrained weights	60
5.4	Comparison of mAP of models	62
5.5	Comparison of f1 score of models	63
5.6	Comparison of inference time of models	64
5.7	Analysis of impact of resolution on inference time and mAP	65
6	Conclusion and Future work	67
6.1	Conclusion	67
6.2	Lessons learned	67
6.3	Future work	68
	Bibliography	69

List of Figures

2.1	Processed Image from DeepLesion dataset[89] with bounding box of size 256x256 pixels	6
2.2	Processed image from DeepLesion dataset[89] showing impact of aspect ratio on bounding box	9
2.3	Processed image from DeepLesion dataset[89] showing impact of scales on bounding box	9
2.4	PANet Architecture [48]	10
2.5	Intersection over Union (IoU) [58]	12
2.6	Precision and Recall [81]	14
2.7	The figure shows 24 detections represented by red bounding boxes and 15 ground truths represented by the green bounding boxes [58]	16
2.8	Precision and Recall values calculated for IOU threshold 0.3 [58] . .	17
2.9	Precision x Recall curve for All-point interpolation technique [58] .	18
2.10	Precision x Recall curve for 11-point interpolation technique [58] . .	18
3.1	height and width distribution of bounding box of the lesion [88] . .	22
3.2	Focal loss [45]	27
3.3	RetinaNet architecture [45]	28
3.4	Landmarks and bounding box in a cardiac MR volumetric image [50]	29
3.5	Architecture of One-stage Multi-task Detector for 3D Cardiac MR Imaging [50]	30
3.6	Overview of detection process of RCNN [26]	32
3.7	Fast-RCNN overview [24]	33
3.8	RPN architecture [67]	35
3.9	Faster RCNN architecture [67]	36
3.10	Main idea of R-FCN [10]	37

3.11	Deeplung framework [96]	38
3.12	3D Aggregated Faster R-CNN architecture [93]	39
3.13	mAP of various models on COCO [65]	40
4.1	Image from DeepLesion dataset[89] before and after processing . . .	44
4.2	Subnetworks of RetinaNet model [45]	47
4.3	Working of YOLO model [66]	49
4.4	YOLO Architecture [85]	49
4.5	Comparison results of YOLOv5 [35]	54
5.1	mAP plot	57
5.2	Precision,Recall,F1 score plot	57
5.3	YOLOv5 models plot for mAP	58
5.4	YOLOv5 models plot for f1 score	59
5.5	Comparison of mAP models with and without initialization weights	60
5.6	Comparison of f1 score of models with and without initialization weights	61
5.7	Comparsion of mAP of models at different resolutions	62
5.8	Comparison of f1 score of models at different resolutions	63
5.9	Comparison of inference time of models at different resolutions . . .	64

List of Tables

2.1	Sample Anchor Parameters for RetinaNet model[45]	8
3.1	Image size distribution of DeepLesion dataset[89]	24
3.2	Train Validation Test split for DeepLesion dataset[89]	24
5.1	Impact of Resolution on inference time and mAP	65

Introduction

Object detection has captivated a lot of attention in recent years because it has a vast range of applications and technological breakthroughs. The task of object detection has been explored extensively in academia as well as real-world applications like monitoring security, transportation surveillance, autonomous driving, drone scene analysis, robot perception, and more[33]. In this project, our work is focused on object detection in dense volume data. One of the types of dense volume data is Computed Tomography (CT) scan.

CT scans are not limited to the medical domain; they are also used in industries. In industries, they are used for the detection of flaws like cracks and voids as well as particle analysis of materials. They are used in metrology for the measurement of internal and external geometry of complex parts. According to the iData Researchs medical imaging procedures analysis, over 75 million CT scans are performed each year in the United States alone. This number is forecasted to reach 84 million procedures by 2022[32].

Analysis of CT scans for the diagnosis of the disease is a tedious task and requires a lot of human effort and working hours, and a small human error in the diagnosis could put the patient's life to risk. So to minimize this risk, a lot of research is being done to perform automatic as well as semi-automatic diagnosis of CT scans. In platforms like Kaggle, we can find competitions like RSNA pneumonia detection challenge[57], COVID 19 CT scans[37] where they provide labeled data to solve the problem of automatic diagnosis of CT scans. The

datasets like DeepLesion dataset[89], covid-19-chest-x-ray-lung-bounding-boxes-dataset[6] have been provided by the medical institutes to openly involve people to develop systems to perform object detection in CT scans. One of the factors that impact both the accuracy as well as the speed of the object detection models is the resolution of the image[51]. Hence in this project, we intend to look deeper into the impact of resolution on the overall performance of the model.

1.1 Problem statement

Our project focuses on object detection in dense volume data. One of the types of dense volume data is Computed Tomography (CT) scan. CT scans have several applications. They are used in the medical domain. They are used in airport baggage screening, assembly lines, and the object detection systems in these places should be able to detect objects fast. Low-resolution CT scanning is fast. Using low-resolution image is one of the ways to address the issue of computational complexity and making the object detection systems fast. Hence using the low-resolution image is helpful in making the entire process of scanning as well as detection faster. Even in the medical domain, to reduce the radiation dose, the exposure time of the patient should be reduced[69]. The exposure time of patients could be reduced by allowing low-resolution CT scans. So, finding out which object detection model has better accuracy as well as speed at low-resolution CT scans is quite relevant. But the existing approaches didn't provide details about how the model would perform when the resolution of CT scans is varied. Hence in this project, our goal is to analyze the impact of varying resolution of CT scans on both the speed and accuracy of the model. We trained three object detection models, namely RetinaNet[45], YOLOv3[65], and YOLOv5[35], at various resolutions on DeepLesion dataset[89].

Theoretical Background

In this section, first, the various types of 3D data are discussed. In this project, we are working on resolution. Hence brief introduction to resolution is provided. As the resolution is varied, the size of the bounding box changes. Thus anchor optimization becomes inevitable. So in this chapter, a brief introduction to the anchor parameters is also provided. Some of the architectures of CNN like Path Aggregation Network (PANet)[48], and Cross Stage Partial Network (CSPNet)[82] which are used in this project, are also explained briefly.

2.1 Datasets

Datasets have been consequential in the domain of object detection. Datasets are not only common ground to measure and compare the capability of algorithms but also pushed the field towards improving the system so that complex problems of object detection could be solved. In this era, deep learning methods have been very successful in several visual recognition problems. It could not have been possible without the accessibility of vast amount of labeled data[47].

The datasets for 3D object detection can be broadly classified into two categories:

- Sparse 3D point cloud
- Dense volume data

2.1.1 Sparse 3D point cloud

The sparse 3D point clouds could be classified into two categories, namely Light Detection and Ranging (LiDAR) data and RGB-D images. In comparison to RGB-D images, LiDAR data is exceptional. On the one hand, LiDAR data provides structural and spatial information of relative location and precise depth. On the other hand, LiDAR data is sparse, unordered, and locality sensitive, and hence it becomes more challenging to process raw LiDAR data[91]. Both data types are discussed in the following sections.

LiDAR data

LiDAR is a remote sensing technique in which the distance to the target is gauged by illuminating the target utilizing a laser light and using a sensor in order to measure the reflection of light[63]. The difference in wavelengths and laser return times are then used to obtain a 3D representation of the target. LiDAR data has several applications in surveying, geomatics, geography, archaeology, seismology, forestry.

RGB-D data

RGB-D data is a combination of RGB data along with its corresponding depth data. One of the RGB-D object detection datasets is SUN RGB-D[72].

2.1.2 Dense volume data

Dense volume data is also referred to as dense 3D data in [16]. Dense volume data like Computed Tomography (CT), Positron Emission Tomography (PET), Magnetic Resonance Imaging (MRI) are arranged on the cartesian voxel grid. The data values among the grid points are typically computed using trilinear interpolation within voxel cells[61]. Some of the types of dense volume data are discussed below.

Computed Tomography (CT)

The term Computed Tomography refers to the X-ray imaging process. In this process, a narrow beam of X-rays is aimed at the patient and rotated quickly about the body. This rotation produces signals. These signals are utilized by the computer of the machine to get cross-sectional images or slices of the body. These slices are referred to as tomographic images. In comparison to conventional X-rays, tomographic images provide more comprehensive information [55]. The principle remains the same when CT scans are used for other objects as well. CT scans are not limited to the medical domain; they are also used in industries. In industries, they are used for the detection of flaws like cracks and voids as well as particle analysis of materials. They are used in metrology for the measurement of internal and external geometry of complex parts. Some of the CT scan object detection datasets available are RSNA pneumonia detection challenge[57], COVID 19 CT scans[37], DeepLesion dataset[89], covid-19-chest-x-ray-lung-bounding-boxes-dataset[6].

Magnetic Resonance Imaging (MRI)

“Magnetic Resonance Imaging (MRI) is a non-invasive imaging technology that produces three-dimensional detailed anatomical images.”[56] It is used for the diagnosis and detection of diseases as well as monitoring the treatment of patients. The technology used in MRI is quite sophisticated. There are protons present in the water, which make up the living tissues. The change in the direction of the axis of rotation of these protons is monitored and excited by MRI. MRI scanners are proficient in getting the image of soft tissues and non-bony parts of the body. They are different from CT scans in the way that they do not use damaging ionizing radiation of the X-rays. It instead uses powerful magnetic field[56]. In [94, 8], they use MRI data as an input for the task of object detection.

2.2 Resolution of image

Resolution means the number of pixels present in an image. The width and height of the image can be used to determine the resolution of image. Resolution also refers to the total pixels an image contains. If an image is 2048 pixels high and 1536 pixels wide.

Total pixels = $2048 \times 1536 = 3145728$ pixels or 3.1 Megapixels

So the resolution image is 3.1 Megapixels[54]. One of the factors that impact the performance of object detection models is the resolution of images[51]. One of the advantages of using low-resolution images is that it helps to reduce the computational complexity of the model. The frames per second the model can predict also increases when low-resolution image is used. But downsampling image leads to information loss and affects the accuracy of the model[70].

2.3 Anchor Parameters

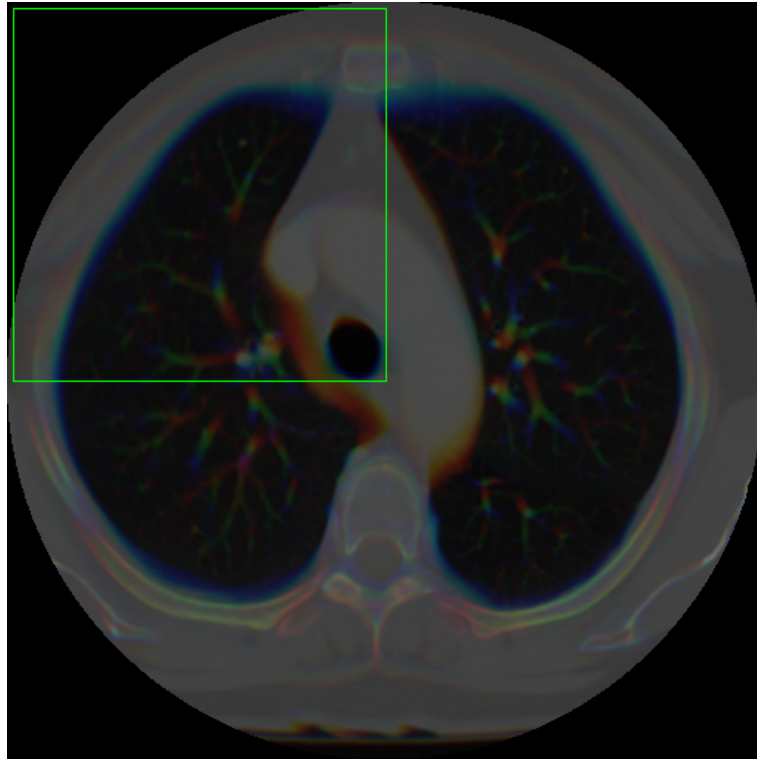


Figure 2.1: Processed Image from DeepLesion dataset[89] with bounding box of size 256x256 pixels

Object detection focuses to find the location of objects as well as predict the related class labels in the image. In simple words, the task of object detection is to detect all the instances of objects present in an image and place bounding boxes around those objects. Early object detection architectures were built as the

ensemble of feature extractors, which are hand-crafted like Histogram of Oriented Gradients(HOG)[11], Viola-Jones detector[79] etc. These models were inaccurate, slow, and performed badly on unfamiliar datasets [92]. In deep learning-based object detection techniques, anchor boxes are primary components that serve as initial suggestions of bounding boxes to detect objects. An extensive set of distributed anchors with pre-defined aspect ratios and scales is sampled evenly over the feature maps. The position offsets relative to the anchors, shape offsets, and classification confidence are forecasted using a neural network[52]. The anchor parameters are discussed in this section.

The various anchor parameters are listed below:

- **Sizes** : Size represents the size of the bounding box used. The data in the Table 2.1 is based on these sources [97, 45]. In the Table 2.1 we can see that there are various sizes 16,32,64,128 and 256. If the scale and ratio are fixed to one and size is 256, the bounding box generated is 256^2 , i.e., 65536 pixels. This is well illustrated in Figure 2.1.
- **Strides**: Stride is a parameter of the filter used in the convolutional neural network, which determines the amount of movement over the video or image. If the stride is fixed to 1, then the given filter will move one pixel at a time. The size of the filter greatly impacts the encoded output volume. In object detection, the anchor box is the filter, and it moves across the image with a given stride to detect the objects. In Table 2.1 it is observed that the value of strides used is 8,16,32,64 and 128[12].
- **Ratio**: The aspect ratio of a given image (referred to as ratio in Table 2.1) is the ratio of its width and height. It is usually represented using two numbers that are separated by a colon. If the aspect ratio of an image is p:q, then it means that the image is p units wide and q units high[1]. In Table 2.1 we can see that the aspect ratios used are 0.5,1 and 2. Suppose bounding box of size 256×256 , and the aspect ratio is fixed to 0.5. Then $w/h = 0.5$, this means $w = 0.5h$. Now the area of the bounding box is

$$A = 256 * 256 \text{ pixels}$$

$$A = 65536 \text{ pixels}$$

$$w * h = 65536 \text{ pixels}$$

$$0.5h * h = 65536 \text{ pixels}$$

$$0.5 * h^2 = 65536 \text{ pixels}$$

$$h^2 = 65536 / 0.5 \text{ pixels}$$

$$h^2 = 131072.0 \text{ pixels}$$

$$h = \sqrt[2]{131072.0} \text{ pixels}$$

$$h = 362.03867196751236 \text{ pixels}$$

$$w = 0.5 * h \text{ pixels}$$

$$w = 0.5 * 362.03867196751236 \text{ pixels}$$

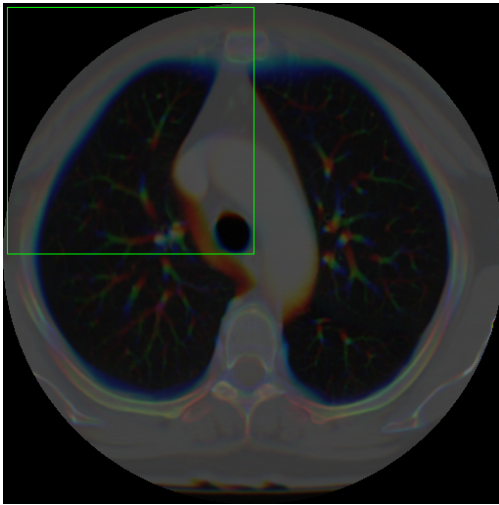
$$w = 181.01933598375618 \text{ pixels}$$

Therefore for a bounding box of size 256 and aspect ratio of 0.5, the dimensions of the bounding box are 181×362 .

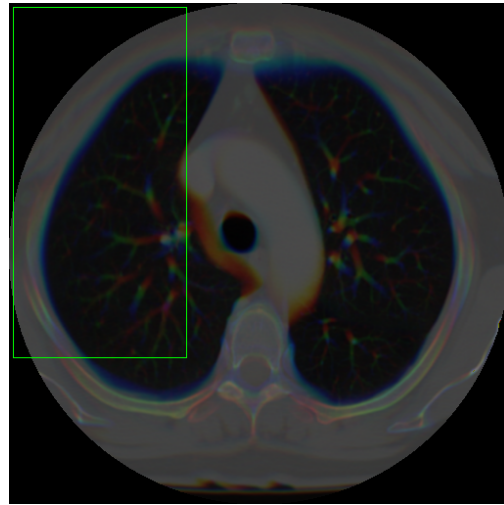
- **Scales:** The size of the bounding box can be scaled using the parameter scale. If the size of the bounding box used is 256 and the scale used is 0.5, then the height of the bounding box is $0.5 * 256$, i.e., 128. Similarly, the width of the bounding box is also 128. The overall area of the bounding box is $128 * 128$, i.e., 16384. If the scale used was 1, the area would have been $256 * 256$, i.e., 65536 pixels. If the ratio of the area with a scaling of 1 and the area is taken with a scaling of 0.5, then the output is $65536 / 16384 = 4$. This phenomenon is well illustrated in Figure 2.3. In Figure 2.3a the size of the bounding box at scale 1 is illustrated. Moreover, in Figure 2.3b the size of the bounding box is reduced by four times when the scale of 0.5 is used.

Parameters	values
sizes	[16, 32, 64, 128, 256]
strides	[8, 16, 32, 64, 128]
ratios	[0.5, 1, 2]
scales	[0.85, 1.08, 1.36]

Table 2.1: Sample Anchor Parameters for RetinaNet model[45]

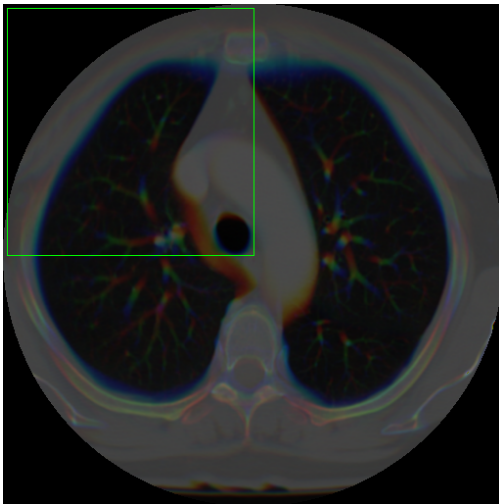


(a) aspect ratio = 1 [89]

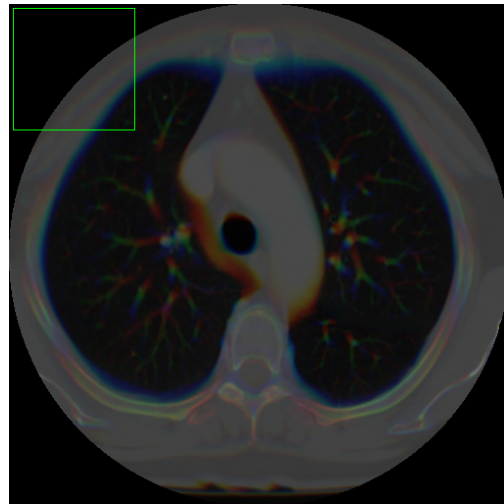


(b) aspect ratio = 0.5 [89]

Figure 2.2: Processed image from DeepLesion dataset[89] showing impact of aspect ratio on bounding box



(a) scale = 1 [89]



(b) scale = 0.5 [89]

Figure 2.3: Processed image from DeepLesion dataset[89] showing impact of scales on bounding box

2.4 Architectures of CNN

In this section the two architectures of CNN, Path Aggregation Network[48] and Cross Stage Partial Network[82] which are used in this project are elaborated in succinct way.

2.4.1 Path Aggregation Network (PANet) [48]

The way the propagation of information takes place in neural networks is quite consequential. So to boost the flow of information in the instance segmentation network, Lie et al. [48] introduced Path Aggregation Network (PANet). This approach demonstrates that the propagation of information in Mask R-CNN[30] (which is CNN architecture used for pixel-level instance segmentation) can be enhanced. A long path between topmost features and low-level structures makes access to precise localization information hard. This entire process can be improved since the information which is dropped in other levels is useful for final prediction. In the end, mask prediction is done on an individual view, and the opportunity to collect more diversified information is lost. PANet tries to address this issue.

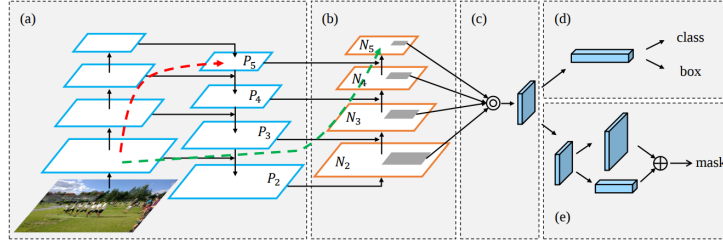


Figure 2.4: PANet Architecture [48]

The architecture of PANet can be seen in Figure 2.4. Bottom-up path augmentation is added in lower layers to shorten the information propagation path and improve the feature pyramid network with signals of accurate localization. Adaptive feature pooling is developed to enhance the information path that exists between every proposal and feature level. It is a component that aggregates features from different feature levels for every proposal. This also avoids arbitrarily assigned results. Mask prediction is augmented with fully connected layers to

capture multiple views of each proposal. The two views are fused to increase the information diversity and produce masks with better quality.

2.4.2 Cross Stage Partial Network (CSPNet) [82]

Wang et al. [82] introduced Cross Stage Partial Network (CSPNet). CSPNet architecture reduces the computational cost and enhances the accuracy and inference speed. The main idea is to propagate the gradient flow through several network paths by partitioning the gradient flow. CSPNet addresses following 3 problems:

- **Reducing memory costs:** Dynamic Random-Access Memory (DRAM) has a high wafer fabrication cost. The cost of Application-Specific Integrated Circuit(ASIC) can be reduced significantly if DRAM costs could be reduced. Hence to reduce the use of memory, cross channel pooling[27] is adopted during the period when feature pyramids are generated. CSPNet reduces the memory usage of PeleeNet[83] by 75 percent while generating feature pyramids.
- **Removing computational bottlenecks:** Very high computational bottleneck results in more cycles to finish the process of inference. This results in making some arithmetic units to often remain idle. This paper tries to spread the computational cost evenly on every layer of CNN so that the rate of utilization of every computation unit could be upgraded and minimize the energy consumption. The computational bottleneck of PeleeNet is reduced to half by using CSPNet.
- **Enhancement of learning ability of a CNN:** The accuracy of CNN is degraded by lightweightening. The proposed CSPNet is applied to ResNeXt[86],DenseNet[31] and ResNet[29]. This led to a reduction of computation effort from ten to twenty percent, and it outperformed ResNet, ResNext, DenseNet, HardNet in terms of accuracy of ImageNet classification task.

2.5 Evaluation metrics

This section explains various evaluation metrics like Intersection over Union, Precision, Recall, Precision x Recall curve, Mean Average Precision, and F1 score.

The explanation is based on these sources [58, 62].

2.5.1 Intersection over Union (IoU)

IoU is just an evaluation metric. Any algorithm that has bounding box coordinates as output can be assessed using IoU. IoU measures the overlap between the predicted bounding box and the ground-truth bounding box. The procedure to compute IoU between the predicted bounding box and the ground-truth bounding box is illustrated in Figure 2.5.

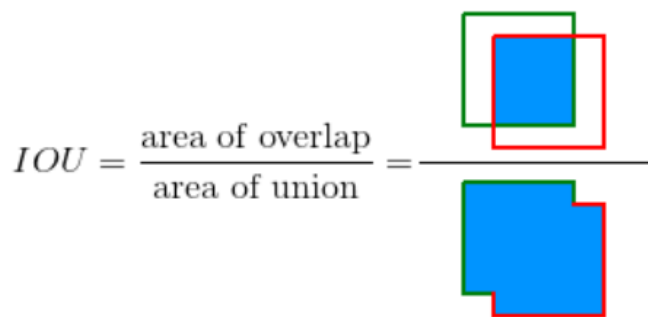


Figure 2.5: Intersection over Union (IoU) [58]

As seen in Figure 2.5, IoU is the ratio of the area of overlap and area of union. If the IoU score is more than 0.5, then it is considered a good prediction. The terms like True Positive, False Positive, and False Negative used in the field object detection are explained briefly below:

- True Positive: A prediction is deemed as True Positive if the IoU is greater than or equal to the given threshold.
- False Positive: A prediction is deemed as False Positive if the IoU is less than the given threshold.
- False Negative: A ground-truth is deemed as a False Negative if it is undetected.

2.5.2 Precision

Precision or positive predictive value is a metric used to gauge an algorithm's proficiency in identifying or categorizing the relevant objects. In simple words, it helps us find how much percentage of all the predictions made by the algorithm is correct. Mathematically precision can be represented by the formula,

$$\text{Precision} = \frac{TP}{TP + FP} \quad (2.1)$$

2.5.3 Recall

Recall, also referred to as sensitivity, is a metric used to gauge the proficiency of an algorithm for categorizing or identifying all the relevant objects. In plain words, it helps us to find how much percentage of all the known cases that are relevant are correctly predicted by the algorithm. Mathematically it is represented by the formula:

$$\text{Recall} = \frac{TP}{TP + FN} \quad (2.2)$$

2.5.4 F1 score

F1 score, also referred to as Dice similarity coefficient (DSC), is the harmonic mean of precision and recall. Mathematically it is represented by the following formula:

$$\begin{aligned} F_1 &= \frac{2}{\frac{1}{\text{recall}} + \frac{1}{\text{precision}}} = 2 \times \frac{\text{precision} \times \text{recall}}{\text{precision} + \text{recall}} \\ &= \frac{2tp}{tp + \frac{1}{2}(fp + fn)} \end{aligned} \quad (2.3)$$

Harmonic Mean penalizes the extreme values. In most real-world classification problems, the class is imbalanced, and the F1 score is instrumental in these cases.

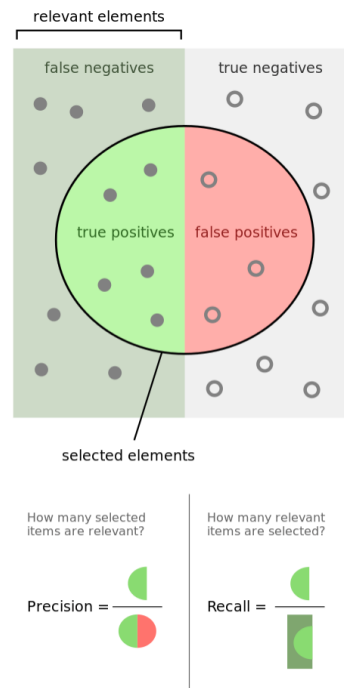


Figure 2.6: Precision and Recall [81]

2.5.5 Precision x Recall curve

The Precision x Recall curve (PR curve) is used to assess the capability of an object detection model. An object detection model is deemed good if its precision remains high as the recall rises. This means that if the confidence threshold is varied, both precision, as well as recall would still be high. Another way to assess if the given object detection model is good is to ascertain if the object detection model identifies only relevant objects (high precision = 0 FP) and finds all ground-truth objects (high recall = 0 FN). A flawed object detection model has to increase the number of objects that are detected (increase FP leading to lower precision) in order to detect all the ground truth objects (higher recall) [58]. If the area under the curve (AUC) is high, it indicates that both recall, as well as precision are high.

2.5.6 Average Precision(AP)

Average precision is computed as the AUC of the PR curve. AP can be computed in two ways. They are:

- 11-point interpolation
- All-point interpolation

11-point interpolation

In this technique, the precision x recall curve shape is obtained by computing the mean of the highest precision values for eleven equally spaced recall levels, i.e. $[0,0.1,0.2,0.3,\dots,1]$ which is given by:

$$AP_{11} = \frac{1}{11} \sum_{R \in \{0,0.1,\dots,0.9,1\}} P_{\text{interp}}(R) \quad (2.4)$$

where,

$$P_{\text{interp}}(R) = \max_{\tilde{R}: \tilde{R} \geq R} P(\tilde{R}) \quad (2.5)$$

All-point interpolation

In our project we are using the all-point interpolation. In all-point interpolation one interpolates through all points, it is given by the following formula:

$$AP_{\text{all}} = \sum_n (R_{n+1} - R_n) P_{\text{interp}}(R_{n+1}) \quad (2.6)$$

$$P_{\text{interp}}(R_{n+1}) = \max_{\tilde{R}: \tilde{R} \geq R_{n+1}} P(\tilde{R})$$

AP is calculated by interpolating the precision value at each level and picking the highest precision whose value of recall is larger than or equal to R_{n+1} .

Practical Example

The calculation of AP using both all-point interpolation and 11-point interpolation can be well illustrated with a practical example. The example has been taken from

the paper [58]. AP is computed separately for each class. In the example which is displayed in Figure 2.7, the red bounding boxes represent prediction made by the object detection model, and they are represented by letters A, B..Y. The green bounding boxes represent the ground truth. The percentage value that is shown next to each red bounding box is the confidence or score for that particular object class. There are altogether 24 detections and 15 ground truths for the seven images shown in Figure 2.7. To get the precision and recall value, the IoU threshold of 0.3 has been selected. Figure 2.8 shows the detections that are ordered with respect to the confidence level. Figure 2.8 is used to obtain the precision-recall curve for all-point interpolation as shown in Figure 2.9. Using the formula

$$AP_{\text{all}} = \sum_n (R_{n+1} - R_n) P_{\text{interp}}(R_{n+1}) \quad (2.7)$$

$$\begin{aligned} AP_{\text{all}} &= 1 * (0.0666 - 0) + 0.6666 * (0.1333 - 0.0666) \\ &\quad + 0.4285 * (0.4 - 0.1333) + 0.3043 * (0.4666 - 0.4) \\ AP_{\text{all}} &= 24.56\% \end{aligned} \quad (2.8)$$

Thus AP is calculated using all-point interpolation.

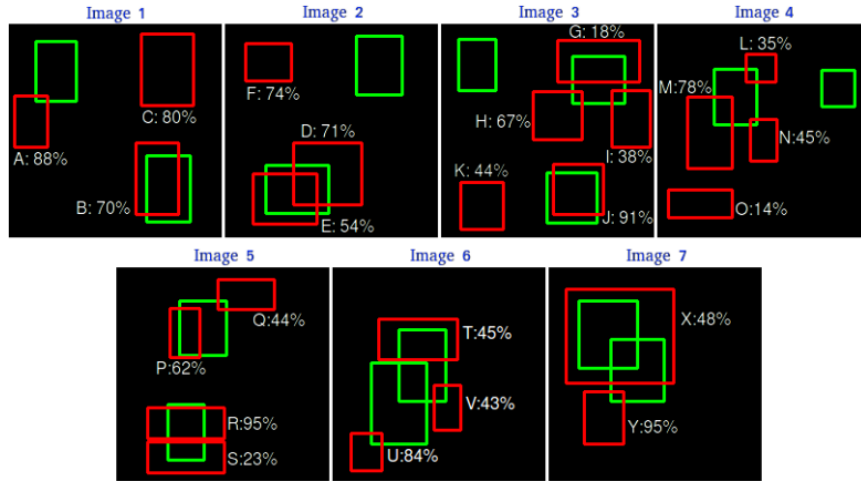


Figure 2.7: The figure shows 24 detections represented by red bounding boxes and 15 ground truths represented by the green bounding boxes [58]

Similarly, by the 11-point interpolation technique, AP can be computed as

detection	confidence	TP	FP	acc TP	acc FP	precision	recall
R	95%	1	0	1	0	1	0.0666
Y	95%	0	1	1	1	0.5	0.0666
J	91%	1	0	2	1	0.6666	0.1333
A	88%	0	1	2	2	0.5	0.1333
U	84%	0	1	2	3	0.4	0.1333
C	80%	0	1	2	4	0.3333	0.1333
M	78%	0	1	2	5	0.2857	0.1333
F	74%	0	1	2	6	0.25	0.1333
D	71%	0	1	2	7	0.2222	0.1333
B	70%	1	0	3	7	0.3	0.2
H	67%	0	1	3	8	0.2727	0.2
P	62%	1	0	4	8	0.3333	0.2666
E	54%	1	0	5	8	0.3846	0.3333
X	48%	1	0	6	8	0.4285	0.4
N	45%	0	1	6	9	0.4	0.4
T	45%	0	1	6	10	0.375	0.4
K	44%	0	1	6	11	0.3529	0.4
Q	44%	0	1	6	12	0.3333	0.4
V	43%	0	1	6	13	0.3157	0.4
I	38%	0	1	6	14	0.3	0.4
L	35%	0	1	6	15	0.2857	0.4
S	23%	0	1	6	16	0.2727	0.4
G	18%	1	0	7	16	0.3043	0.4666
O	14%	0	1	7	17	0.2916	0.4666

Figure 2.8: Precision and Recall values calculated for IOU threshold 0.3 [58]

follows:

$$\begin{aligned} \text{AP}_{11} &= \frac{1}{11}(1 + 0.6666 + 0.4285 + 0.4285 + 0.4285) \\ \text{AP}_{11} &= 26.84\% \end{aligned} \tag{2.9}$$

This calculation is well illustrated in Figure 2.10. It can be seen that the results obtained by using all-point interpolation and 11-point interpolation techniques are different.

2.5.7 Mean Average Precision (mAP)

Mean average precision(mAP) is one of the most popular metrics used for measuring the performance of object detection models. Several variants of mAP are used by different object detection challenges, namely PASCAL VOC[17], MS COCO[42]. The mAP can be defined as the average AP over all the classes, it is

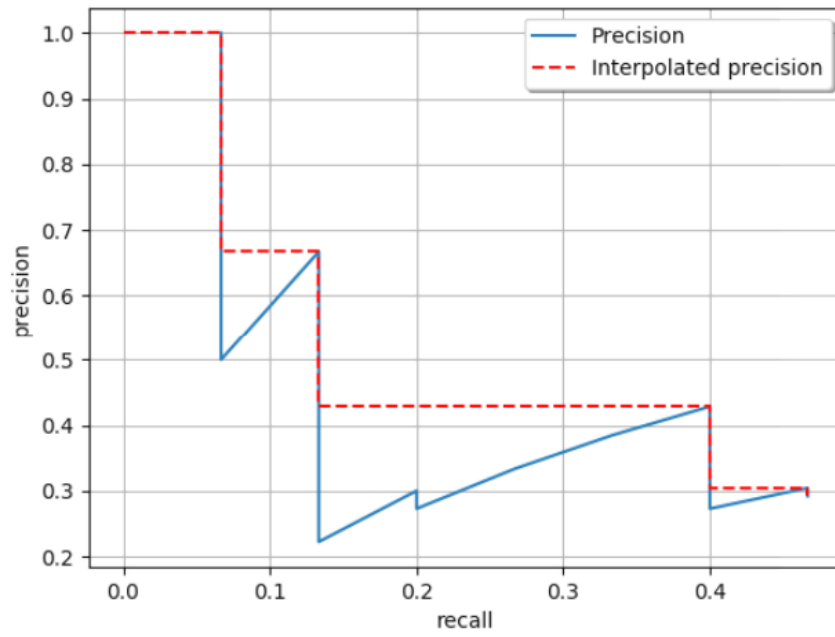


Figure 2.9: Precision x Recall curve for All-point interpolation technique [58]

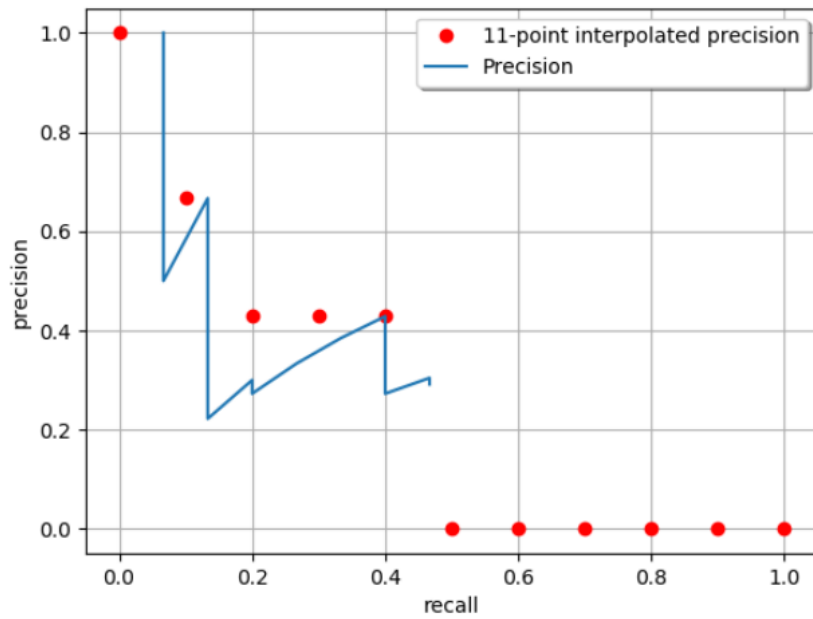


Figure 2.10: Precision x Recall curve for 11-point interpolation technique [58]

calculated using the following formula:

$$\text{mAP} = \frac{1}{N} \sum_{i=1}^N \text{AP}_i \quad (2.10)$$

Related work

In this section, an overview of publicly available CT scan datasets is provided. Then DeepLesion dataset is discussed. After that an overview of object detection is provided. Then the approaches of object detection, namely one stage object detection and two-stage object detection are discussed. A summary of the object detection models discussed is provided.

3.1 CT scan Datasets

Some of the publicly available CT scan datasets are as follows:

- RSNA pneumonia detection challenge [57]: The dataset contains 29684 Digital Imaging and Communications in Medicine (DICOM) images. The bounding box coordinates, patient id, and the target are provided as data fields. The dataset is of 2D CT scans. Hence it cannot be used in this project.
- COVID 19 CT scans [37]: This dataset consists of 20 3D CT scans. The annotation is for the task of segmentation. The dataset is quite small and cannot be used.
- DeepLesion dataset [89]: This dataset consists of 32120 slices of 3D CT scans. The bounding box coordinates, patient id, target, sex, etc., are provided as the data field. The dataset is suitable for training deep learning models. Hence this dataset is suitable for this project.

- covid-19-chest-x-ray-lung-bounding-boxes-dataset [6] : This dataset has a total of 616 images. Bounding box coordinates are provided as annotation. The annotations are of COCO format. The dataset also has other information like modality, survival, sex, view. The image was annotated manually by qualified radiologist. The dataset consists of 2D CT scans, hence it cannot be used in this project.

Out of all of the datasets mentioned above, DeepLesion dataset is suitable for our project because it has bounding box coordinates for slices of 3D CT scans and is also large enough for training deep learning model. Hence this dataset was used in this project.

3.1.1 DeepLesion dataset [89]

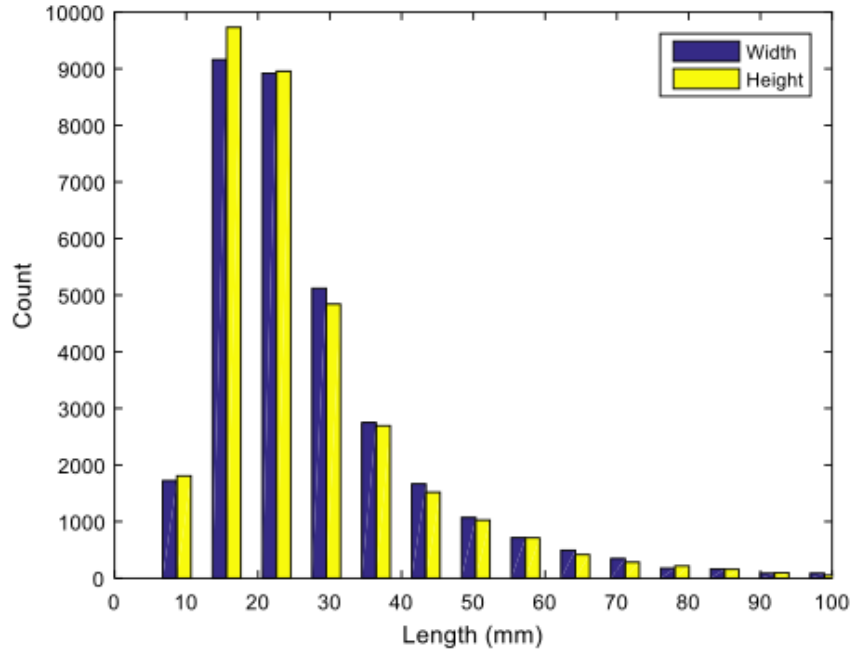


Figure 3.1: height and width distribution of bounding box of the lesion [88]

DeepLesion dataset was released on 19 July 2018. This dataset has 32,120 slices of CT scans which were collected from 10,594 studies conducted on 4,427

patients. Each slice has from 1 to 3 lesions, so there are 32,735 lesions in total. DeepLesion dataset has lesions from eight locations, namely abdomen, bone, lung, liver, pelvis, soft tissue, mediastinum, and kidney, with significant variations of diameters ranging from 0.21 to 342.5mm. The height and width distribution of DeepLesion dataset is shown in Figure 3.1. Most of the current research work focuses on developing a Computer-Aided Detection system that specializes on specific lesion types. For example, Brain Tumor Segmentation [53] challenge emphasizes brain lesion segmentation. Another dataset Lung Nodule Analysis [4] challenge, focuses on lung nodule detection. However, this dataset helps in developing object detection systems suitable to localize and classify multiple lesion types throughout the body. The variation of the lesion size is beneficial for finding out which object detection model is more robust and detects objects of variable sizes.

Annotations

The lesions are labeled by utilizing response evaluation criteria in solid lesions (RECIST) diameters that contain two lines. The first line gives the measurement of the lesion’s longest diameter, and the second line offers the measurement of the lesion’s longest perpendicular diameter along the measurement plane. Bounding boxes that are provided have been produced from these RECIST diameters. For every lesion, there are thirty millimeters of additional slices below and above the key slice that provides the contextual information. Typically images have three channels, but the image provided in this dataset are single-channel. Usually, each channel is 8 bit, but in this dataset, the channel is 16-bit. Most of the images are 512x512 pixels, but there are some exceptions. The distribution of images based on size is shown in Table 3.1.

Seventy percent of the images are labeled as training images, fifteen percent of images are labeled as validation, and fifteen percent are labeled as testing images by the authors of this dataset. This distribution is shown in Table 3.2.

There are supplementary annotations like gender and age of the patient, lesion type, etc. The lesion type is only provided for test and validation data. So this

Image size	Count
512×512	32695
768×768	32
1024×1024	8

Table 3.1: Image size distribution of DeepLesion dataset[89]

Data distribution	Number of images	Percent(approximately)
Train	22901	70
Validation	4889	15
Test	4927	15

Table 3.2: Train Validation Test split for DeepLesion dataset[89]

information was not utilized during training the models.

3.2 Object detection overview

The crux of object detection is to locate as well as classify objects. It utilizes rectangular bounding boxes to locate the detected objects and then classify the category of objects. Object detection is one of the critical areas of computer vision. It has several applications in scientific as well as practical industrial productions like face detection, text detection, pedestrian detection, video detection, logo detection, vehicle detection, medical image detection, and so forth. The current state-of-the-art 2D object detection models use deep convolutional neural networks(CNNs) as their backbone and detection network for extracting features from input images or videos and solve the task of classification and localization, respectively[33].

A significant improvement has been obtained in 3D object detection because of Convolutional Neural Networks (CNNs). The data for object detection in the case of 3D could be divided into sparse 3D point cloud and dense volume data. In the case of 3D point clouds, most of the previous works used to convert the point cloud to volumetric representations and CNNs were generalized to 3D CNNs for the task of object detection. 3D FCN[40] uses 3D CNNs to predict 3D bounding boxes as well as class labels. VoxelNet[95] uses 3D CNNs to encode 3D input volumes to 2D feature maps, and these features are fed to subsequent detection

network. In Vote3Deep[16], the sparsity of 3D volume is utilized to accelerate the 3D convolution.

In [96], the dense volume data is fed to the 3D version of the R-CNN model to get the prediction. In [76], slice of dense volume data, i.e., slice of 3D CT scans, are fed to Mask R-CNN[30] to get the prediction. Hence, in dense volume data, the data is fed directly to the model based on 3D CNNs, or slice of 3D data is fed to 2D object detection, which is also based on CNNs. A major difficulty of these 3D algorithms is that they are computationally expensive[90]. The other problem is that 3D data are pretty challenging to collect, annotate and store and hence there are fewer datasets for deep learning models[19].

The approaches to the task of object detection can be broadly classified into two tasks:

- One stage approach
- Two-stage approach

3.2.1 One stage approach

In the one-stage approach, class probabilities and bounding boxes of objects are predicted utilizing a single-stage network. They do not require region proposal generation as well as post-processing. So, single-stage approaches are fast[28]. The fast speed of single-stage approaches make them suitable for real-time object detection systems. The one-stage approach could be divided into two tasks :

- One stage 2D object detection
- One stage 3D object detection

One stage 2D object detection

One-stage 2D object detection approach refers to networks that forecast class probabilities as well as bounding box offsets from input images with single feed-forward CNN. This process does not involve the generation of a proposal region or post classification. All computation is done using an individual network. Some of the single-stage 2D object detection models are discussed below:

SSD Liu et al. [49] introduced SSD, a single shot detector. The main idea behind SSD is to predict box offsets and category scores for a specific set of bounding boxes. These bounding boxes use small convolutional filters that are applied to feature maps. Predictions of multiple scales are produced from feature maps of various scales. These predictions are separated with respect to aspect ratio to get high detection accuracy. These design features help to achieve high accuracy even on images of low resolution, which further enhances the speed vs. accuracy trade-off.

In [22], some drawbacks of SSD have been mentioned:

- SSD cannot detect small objects as the shallow layers in the network may not produce sufficient high-level features to predict small objects.
- SSD requires complex data augmentation to train, suggesting that it needs a huge amount of data for training
- The design of prior boxes impacts the model's performance, but the model does not provide any mechanism to generate the prior boxes.

YOLO Redmon et al. [66] introduced YOLO, also referred to as YOLOv1. The task of object detection is framed as a regression problem for bounding boxes that are separated spatially and the related class probabilities. The bounding box coordinates and class probabilities are forecasted directly from entire images in one stage. Since the task of detection is framed as a regression problem, it does not require a complex pipeline. Hence YOLO is quite fast.

Disadvantages of YOLOv1[23]:

- Struggles to detect objects that are close as each grid proposes only two bounding boxes
- Small objects are not detected efficiently.

YOLOv2[64], YOLOv3[65] and YOLOv5[35] are the subsequent enhanced versions of the YOLO architecture. Some of the applications of YOLO architecture on 3D CT scans are listed below:

- In [2], 2D slices of 3D PET scans are fed to YOLOv2 architecture for automatic localization of normal active organs in 3D PET scans.
- In [46], 2D slices of 3D CT scans are fed to YOLOv3 for automatic detection of pulmonary nodules.
- In [39], 2D slices of 3D CT scans are fed to YOLOv3 for kidney recognition in CT scans.
- In [38], 2D slices of 3D CT scans are fed to YOLOv2 for automatic detection of the nasal cavities and paranasal sinuses.

Retinanet A major problem of one-stage 2D detectors is the extreme imbalance between background and foreground classes. Hence Lin et al. [45] introduced focal loss to tackle this issue. In Figure 3.2 the focal loss function is visualized.

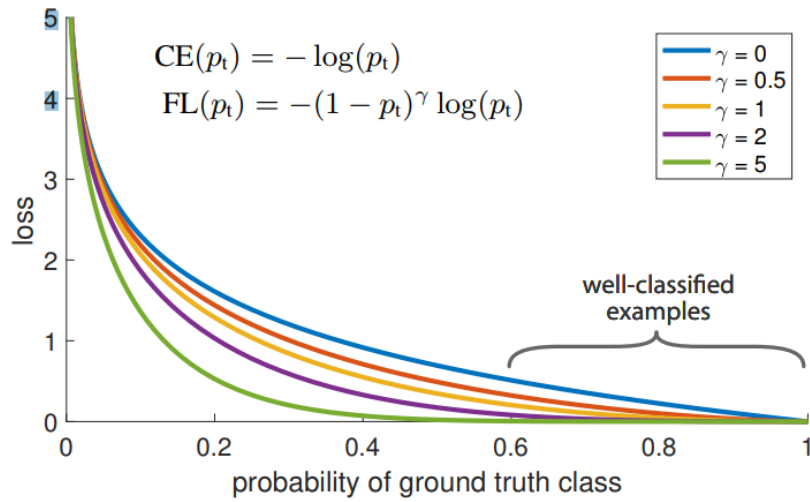


Figure 3.2: Focal loss [45]

The architecture of RetinaNet is shown in Figure 4.2. RetinaNet utilizes Feature Pyramid Network (FPN) [44] as backbone which is placed on top of ResNet architecture [29] to produce multi-scale as well as rich convolutional feature pyramid. RetinaNet combines two subnetworks. One subnetwork is for classifying anchor boxes. The other subnetwork is used for regression. The network design is simple. This paper emphasizes how the focal loss function eradicates the gap of accuracy

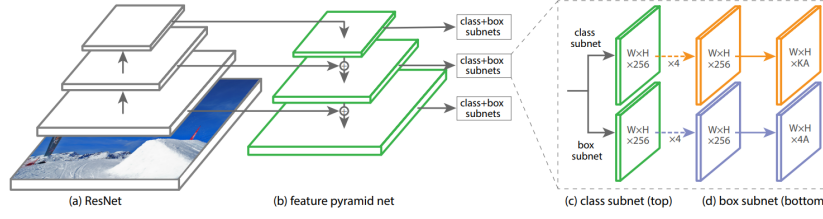


Figure 3.3: RetinaNet architecture [45]

between the one-stage detector and two-stage detectors such as Faster R-CNN[67]. Zlocha et al. [97] had shown that this model gives better results on the DeepLesion dataset than other models like Faster RCNN[67] and Mask RCNN[30]. RetinaNet performs well in the COCO dataset[43]. It has an mAP higher than Faster RCNN on the COCO dataset.

One-stage 3D object detection

Some prominent one-stage 3D object detection techniques are discussed below:

One-stage Multi-task Detector for 3D Cardiac MR Imaging[50] Fast and accurate bounding box detection and landmark localization are consequential for 3D medical imaging. Lu et al. [50] introduced the multi-task framework for real-time, bounding box detection and landmark localization in 3D space. The model extends SSD[49] architecture for this purpose. One cardiac magnetic resonance(MR) volume is composed of 2D image slices that have spatial consistency between them. To learn the 3D context direct approach is to use 3D CNNs. Limited GPU memory and a large number of model parameters present a computational bottleneck for the use of 3D CNNs. To tackle this problem multi-task learning framework referred to as MT3D is proposed. Even though MT3D is based on SSD, it differs from SSD on these aspects:

- SSD is a single-task learning technique, whereas MT3D is multi-task learning.
- SSD detects bounding box and confidence scores, whereas MT3D is tailored to detect bounding boxes and anatomical landmarks from CMR volumes, as shown in Figure 3.4

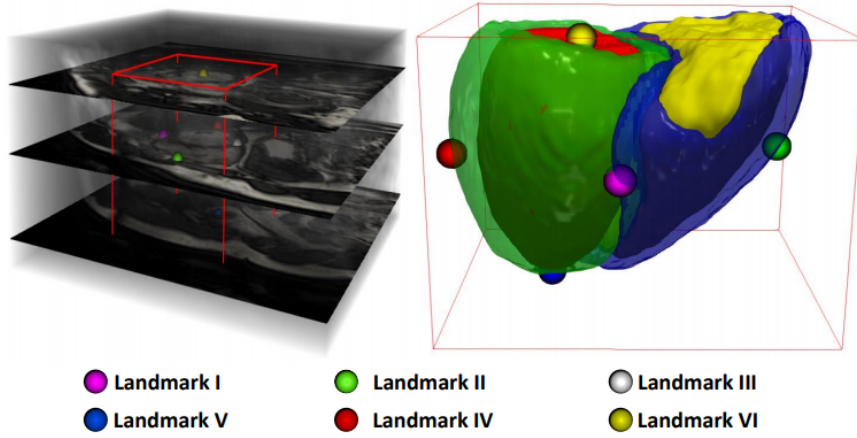


Figure 3.4: Landmarks and bounding box in a cardiac MR volumetric image [50]

- SSD utilizes 2D network implementation, which handles 2D inputs, but MT3D can handle high dimensional problems. 3D MR volumetric image is treated as a multi-channel tensor image. Hence MT3D takes into account the spatial consistency between the slices of image and takes the computational advantage of 2D networks.
- SSD calculates IoU. This IoU is the basis for calculating non-maximum suppression and find the corresponding bounding box. This mechanism is not feasible for landmark selection. Hence MT3D calculates the average location for accurate landmark location by taking the average of network output landmarks.
- SSD utilizes anchor boxes of different sizes, and extra feature layers are appended to the VGG16-like network. This arrangement enabled this model to handle objects of several sizes. The size of the heart does not vary much. Hence in MT3D, the extra feature layers are removed. The last layer of VGG16-like architecture is utilized to predict the landmarks and bounding box. Anchor boxes are also designed to have a square shape to make the problem simple.

Figure 3.5 shows the architecture of the MT3D. It can be seen that MT3D uses the VGG16 as the backbone. It takes the whole cardiac MR image, downsamples

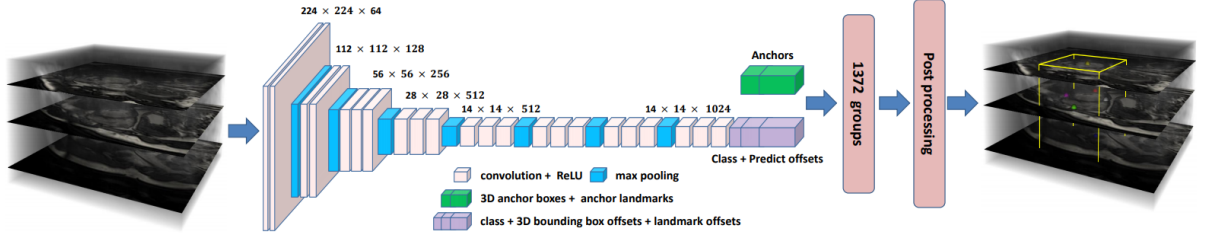


Figure 3.5: Architecture of One-stage Multi-task Detector for 3D Cardiac MR Imaging [50]

the feature maps. Then it predicts the bounding box and landmarks.

3-D Deep Leaky Noisy-OR Network [41] The lesion that occurs in the lung is referred to as pulmonary nodules. With respect to morphology, doctors assess the malignancy of nodules. Different doctors give different results. The accuracy of the results depends on the experience of the doctor. Computer-aided diagnosis (CAD) is appropriate for this task as computer vision models can scan everywhere and are not as prone to tiredness and emotions as humans. Automatic lung cancer detection has many difficulties in comparison to normal computer vision problems. Nodule detection is a 3D object detection task. Generalizing 2D object detection techniques to 3D cases is not feasible because of GPU constraints. So in [60, 14], proposals are extracted using RPN and then combined to produce 3D proposals. The relationship between cancer and nodule is complicated. The presence of nodule does not necessarily indicate lung cancer. Hence nodule detection is the task of multiple instance learning (MIL). So, 3D RPN was developed to predict the bounding box coordinates for nodules. 3D CNN structure helps the model to capture complex features. Because of GPU memory constraints, patches of 3D data are used to train and test the model. Efficient optimization is achieved by training the model end-to-end. To get rid of over-fitting extensive data augmentation is used. The top five suspicious nodules are sent as input to the classifier. The classifier consists of leaky noisy-or model[59] which combines the scores of the top five nodules. There are two significant contributions of this paper. The first one is that they introduce volumetric one-stage-end-to-end CNN for the task of 3D object detection. They propose to use the noisy-or gate in the classifier to solve the task

of MIL.

3.2.2 Two-stage approach

In two-stage approach, several possible regions containing objects are proposed, and then region-wise features to predict the class of each region or proposal are extracted[28].

The two-stage approach could be divided into two tasks :

- Two-stage 2D object detection
- Two-stage 3D object detection

Two-stage 2D object detection

Two-stage 2D object detection approaches are region-based frameworks. In the case of the two-stage approach, region proposals that are category-independent are produced from an image. CNN features are then extracted from these regions. After that, category-specific classifiers are utilized to determine the label of the categories for the proposals[47].

The two-stage 2D object detection algorithms are best represented by the R-CNN family[26, 24, 67, 30]. These two-stage 2D object detection techniques are discussed below:

RCNN

Girshick et al. [26] introduced Region-Based Convolutional Networks (RCNN) that is used for object detection as well as segmentation. It is common to perform the task of image classification by using Histograms of oriented gradients (HOG)[11] or CNN's. In contrast to image classification, object detection focuses on finding objects in images. One way is to address object detection as a regression problem suitable for finding four bounding box coordinates of an individual object. However, it does not help to scale to object of different sizes. Another method could be to utilize the sliding window technique and input that region to CNN. The problem with this method is that all objects have a common aspect ratio and size. To handle these issues, R-CNN is divided into two different modules:

- **Region proposals:** Several papers introduce different techniques for generating region proposals that are category-independent. Some of the examples are objectness [3], constrained parametric min-cuts (CPMC)[7], selective search[77] etc. But R-CNN uses selective search to allow comparison with previous detection networks like [77],[84].
- **Feature extraction:** In this step, a feature vector of fixed-length is extracted from each proposal utilizing a CNN. Region proposal features for a particular region are computed by converting the image data of that region into fixed $S \times S$ pixel size compatible with CNN. Regardless of the aspect ratio and size of the candidate region, all the pixels are wrapped in a tight bounding box to make it a fixed size. Then each extracted feature is scored using Support Vector Machines (SVM)[9] trained for that particular region. Once all the regions in an image are scored, greedy non-maximum suppression is applied for each class independently. The detection process of this model is well illustrated in Figure 3.6.

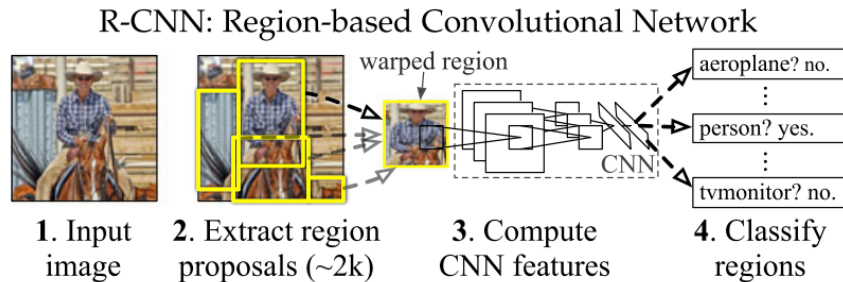


Figure 3.6: Overview of detection process of RCNN [26]

The bounding box regression step was added to extend the design further. After each output has been scored from the CNN using a class-specific linear SVM, the bounding box regressor predicts the new bounding box, which helps achieve a better IoU. The new bounding box is then scored once again to get the final score. This process could be iterated, but the results do not improve much.

Problems with R-CNN [24]

- Training is a multi-stage process, as in the first stage features are extracted, and in the second stage, SVM is used to predict the bounding box.
- Training is expensive in terms of both memory requirements and time.
- Detecting objects is slow as the prediction is done in multiple stages.

Fast RCNN

Girshick et al. [24] proposed Fast R-CNN that builds on R-CNN and addresses the shortcomings of R-CNN. A Fast R-CNN model takes the full image as input. The network processes the entire image with multiple convolutional as well as max-pooling layers to create a convolutional feature map. Then for every object proposal, a feature vector of fixed length is extracted from the feature map by region of interest (ROI) pooling layers. Every feature vector is used as an input for the sequence of fully connected layers that branches into two output layers. The first one generates softmax probability values for the background class and K object classes. The second one considers the K object classes and generates bounding box coordinates for each category. The overview of this model is illustrated in Figure 3.7.

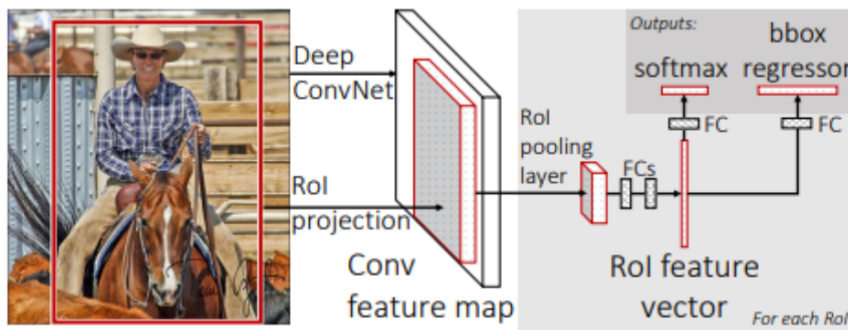


Figure 3.7: Fast-RCNN overview [24]

The major contributions of Fast RCNN are:

- In comparison to R-CNN, it has higher detection quality.
- The training process is single-stage, and it utilizes multi-task loss.

- Training updates all the network layers
- Disk storage is not required to catch features.

Faster RCNN Ren et al. [67] introduced Region Proposal Network (RPN). This model shares the convolutional features of the full image with the network utilized for detection. This sharing of convolutional features makes the generation of region proposals less expensive computationally. The Faster R-CNN model consists of two modules:

- **Region Proposal Network (RPN):** RPN inputs an image of any size and produces a set of object proposals. Each of these proposals also has an objectness score(also referred to as confidence score in many papers). This process is modeled utilizing a fully convolutional network. A small network is slid over the output of the convolutional feature map to generate region proposals. This small network takes the $n \times n$ spatial window of the convolutional feature map as input. All sliding windows are mapped to a low dimensional feature. This architecture is illustrated in Figure 3.8.
- **Fast R-CNN:** The feature map and region proposals generated by RPN are the input for the Fast-RCNN module.

So as a whole first RPN is trained to generate proposals. Then these proposals are used to train Fast R-CNN. Then RPN is initialized by utilizing the network tuned by Fast-RCNN. The entire process is iterated. Fast R-CNN and RPN networks are combined into one network as illustrated in Figure 3.9. In each stochastic gradient descent iteration, region proposals are generated by the forward pass. While training the Fast-RCNN detector, these region proposals are treated like pre-computed, fixed proposals. In the case of backpropagation, the backpropagated signals from Fast R-CNN loss and RPN loss are combined for the shared layers.

Yan et al. [89] used Faster R-CNN for the detection of lesions. The model was trained on the DeepLesion dataset.

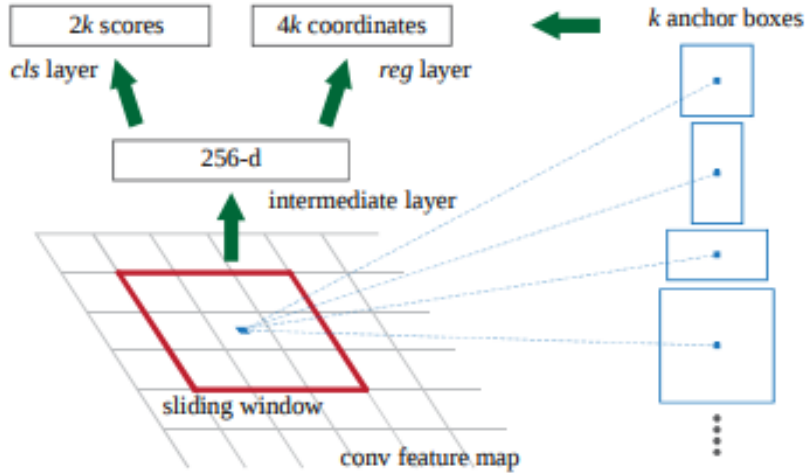


Figure 3.8: RPN architecture [67]

Mask R-CNN

He et al. [30] introduced Mask R-CNN. In terms of architecture, Mask R-CNN has a branch added to predict object mask in Faster R-CNN. The bounding box recognition branch is in parallel with the branch to predict the object mask. The mask branch is little FCN applied to every ROI that predicts the segmentation mask. The segmentation mask is predicted in a pixel-to-pixel manner. Mask R-CNN is simple to implement as well as train since the Faster R-CNN model supports numerous architectural designs. The addition of the mask branch makes the computation slightly expensive. Hence the overall speed of the model is not affected. In Mask-RCNN, to achieve good results, it is inevitable to construct the mask branch efficiently. Pixel-level segmentation needs more fine-grained alignment in comparison to bounding boxes. Hence, in Mask R-CNN, the ROI-pooling layer is enhanced so that region of interest (RoI) can be mapped to the areas of the original image. In [76] Tang et al. used Mask R-CNN to develop a Universal Lesion Detector (ULDor) that was trained on the DeepLesion dataset to detect lesions.

R-FCN Dai et al. [10] introduced R-FCN which is region-based fully convolutional networks for efficient and accurate object detection. R-CNN based detectors process object detection in two stages:

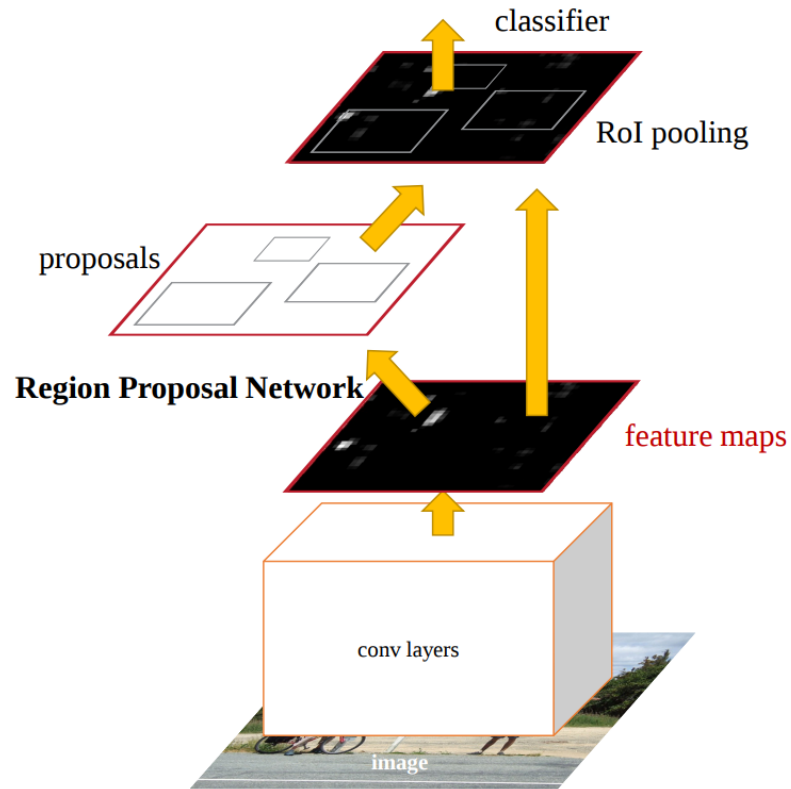


Figure 3.9: Faster RCNN architecture [67]

- region proposals
- region classification

Candidate regions are extracted by RPN[67]. Taking inspiration from [67], they develop the network such that the feature sharing takes place between R-FCN and RPN. This is illustrated in Figure 3.10.

Given the RoIs, the R-FCN network is created to categorize the RoIs into background and object categories. In R-FCN, all trainable weight layers are convolutional and need to be calculated for the full image. This system adopts image classification backbones like ResNets, which are fully convolutional. This method achieves accuracy comparable to Faster R-CNN but is faster while training as well as inference. In [87] Yan et al. introduced 3D context improved region-based CNNs to include 3D context into 2D CNNs to detect lesions from CT scans. Several

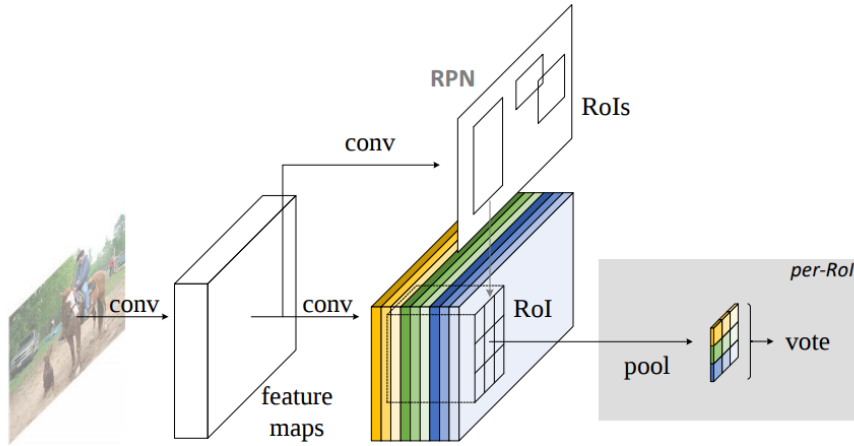


Figure 3.10: Main idea of R-FCN [10]

neighboring slices are fed to enhanced R-FCN to produce feature maps separately. These feature maps are aggregated for final prediction.

Two-stage 3D object detection

Some prominent two-stage 3D object detection models will be discussed here.

Deeplung Zhu et al. [96] introduced Deeplung, a fully automated lung CT cancer diagnosis system. It has two components:

- Nodule detection: To detect the location of candidate nodules
- Classification: To classify the candidate nodules into malignant or benign

The nature of CT data is 3D, and Faster R-CNN is effective for 2D object detection. Hence 3D Faster R-CNN was designed for nodule detection. The architecture also consists of a U-net-like structure and 3D convolutional kernels to effectively learn the latent features. U-net is widely used for semantic segmentation. However, it can capture local and contextual information. Hence it is helpful for nodule detection. The 3D convolutional network has a lot of parameters. Thus it cannot be trained on a public lung CT dataset that has a small size. So, 3D dual-path architecture is used as a building block as the deep dual-path network is compact and has enhanced performance than the deep residual network.

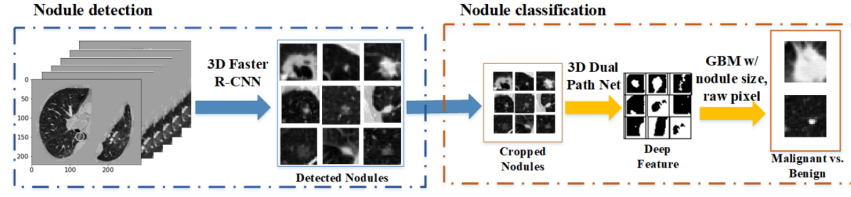


Figure 3.11: Deeplung framework [96]

Hence deep 3D dual-path network is designed to extract features. The performance of Gradient boosting machines (GBM) is boosted when useful features are provided. So GBM is utilized in this network for nodule classification. Thus a fully automatic lung CT cancer detection system is built referred to as Deeplung, it is illustrated in the Figure 3.11.

3D Aggregated Faster R-CNN Zhang et al. [93] introduced an end-to-end 3D Aggregated Faster R-CNN network. This network consists of aggregated classifier branch, which is added to the backbone of RPN. The area of lesions is not uniform, so the region-based classifier branch of Faster R-CNN fails to detect it. Some of the models thus train two kinds of heterogeneous networks. One network is for candidate generation, and another network is for the false-positive rate reduction. This was the motivation behind developing end-to-end 3D Faster R-CNN for the task of lesion detection. As illustrated in Figure 3.12, this network consists of two heads. The first head is RPN and the second one is False Positive Reduction Network(FPRN). Both these heads lie on each pyramid level.

In Figure 3.12 it is also seen that instead of the entire image, cropped image is fed to the network. This is done to fulfill the GPU memory constraint. While training, random crops are used as input to the network. During testing, the detection results obtained from the sliding window pieces are assembled to get the final result. However, this strategy gives rise to a lot of randomness while training and even slows down the convergence speed. To remedy this issue, the authors use the enhanced version of focal loss[45], referred to as adaptive focal loss.

This model was trained on the DeepLesion dataset. 3D CNNs do not work well with the DeepLesion dataset [87]. The authors of this paper give three reasons to

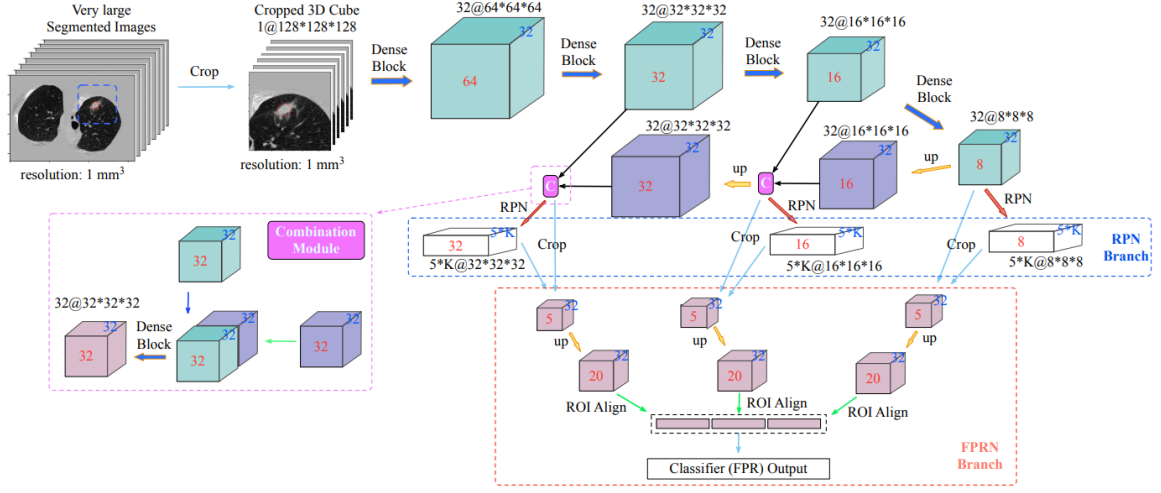


Figure 3.12: 3D Aggregated Faster R-CNN architecture [93]

justify this claim:

- Large slice interval(e.g.,5mm)
- 11 percent of data has large lesions ($\geq 48\text{mm}$), which could easily be out of bound
- The bounding boxes for lesions are larger than the actual size of the lesion.

Bounding box regression becomes a challenge because of these issues. Hence to handle these issues, lesions with more than one annotation is merged, very large lesions($\geq 48\text{mm}$) are eliminated, and the diameter of small lesions is adjusted. The sensitivity obtained at 16 false-positive rate was 94.72. The models were tested with 4 Tesla K80 GPUs (48 GB) with an Intel(R) Xeon(R) E5-2640 v2 CPU, and the inference time was $\tilde{4.2}$ seconds.

3.2.3 Summary

- SSD[49] cannot detect small objects as the shallow layers in the network may not produce sufficient high-level features to predict small objects. In the

	backbone	AP	AP ₅₀	AP ₇₅	AP _S	AP _M	AP _L
<i>Two-stage methods</i>							
Faster R-CNN+++ [5]	ResNet-101-C4	34.9	55.7	37.4	15.6	38.7	50.9
Faster R-CNN w FPN [8]	ResNet-101-FPN	36.2	59.1	39.0	18.2	39.0	48.2
Faster R-CNN by G-RMI [6]	Inception-ResNet-v2 [21]	34.7	55.5	36.7	13.5	38.1	52.0
Faster R-CNN w TDM [20]	Inception-ResNet-v2-TDM	36.8	57.7	39.2	16.2	39.8	52.1
<i>One-stage methods</i>							
YOLOv2 [15]	DarkNet-19 [15]	21.6	44.0	19.2	5.0	22.4	35.5
SSD513 [11, 3]	ResNet-101-SSD	31.2	50.4	33.3	10.2	34.5	49.8
DSSD513 [3]	ResNet-101-DSSD	33.2	53.3	35.2	13.0	35.4	51.1
RetinaNet [9]	ResNet-101-FPN	39.1	59.1	42.3	21.8	42.7	50.2
RetinaNet [9]	ResNeXt-101-FPN	40.8	61.1	44.1	24.1	44.2	51.2
YOLOv3 608 × 608	Darknet-53	33.0	57.9	34.4	18.3	35.4	41.9

Figure 3.13: mAP of various models on COCO [65]

DeepLesion dataset, the lesions are quite small, so it is not suitable for this task.

- In several papers like [2, 46, 39, 38] they use YOLO architecture for the detection objects in slices of 3D CT scans. Hence YOLO architecture could be used to detect lesions in the DeepLesion dataset.
- Zlocha et al. [97] had shown that the RetinaNet model gives better results on the DeepLesion dataset than other models like Faster RCNN and Mask RCNN. In Figure 3.13 it can be observed that RetinaNet performs well in the COCO dataset. It has an mAP higher than Faster RCNN on the COCO dataset.
- In MT3D [50] SSD is enhanced to take high dimensional images as input and predict landmarks and bounding box. The input image is cropped and fed to the network to tackle the constraints of GPU.
- Girshick et al. [26] introduced Region-Based Convolutional Networks (RCNN), a two-stage 2D object detection approach. Training RCNN is a multi-stage process. It is expensive in terms of both memory requirements and time. Ultimately object detection is slow.
- Girshick et al. [24] introduced Fast-RCNN to address the drawbacks of RCNN [26]. In Fast-RCNN training process is single-stage, and it utilizes multi-task loss. Training updates all the network layers, and disk storage is not required to catch features.

- Ren et al. [67] introduced Region Proposal Network (RPN). This network shares the entire image convolutional features with the network used for detection. Yan et al. [89] used Faster R-CNN for the detection of lesions. The model was trained on the DeepLesion dataset.
- He et al. [30] introduced Mask R-CNN. Mask R-CNN is an extension of Faster R-CNN. In terms of architecture, Mask R-CNN has a branch that was added to predict object masks in Faster R-CNN. In [76] Tang et al. used Mask R-CNN to develop a Universal Lesion Detector (ULDor) that was trained on the DeepLesion dataset to detect lesions.
- Dai et al. [10] introduced R-FCN, which is a region-based fully convolutional network for efficient and accurate object detection. This system adopts image classification backbones like ResNets, which are fully convolutional. This method achieves accuracy comparable to Faster R-CNN but is faster while training as well as inference. Yan et al. [87] introduced 3D context improved region-based CNNs to include 3D context into 2D CNNs to detect lesions from CT scans. Several neighboring slices are fed to enhanced R-FCN to produce feature maps separately. These feature maps are aggregated for final prediction.
- Zhu et al. [96] introduced Deeplung, a fully automated lung CT cancer diagnosis system. It uses 3D Faster R-CNN for nodule detection. The CT data is cropped and fed to the network for detection to fulfill the constraints of GPU.
- Zhang et al. [93] introduced an end-to-end 3D Aggregated Faster R-CNN network. While training, random crops are used as input to the network. This model was trained on the DeepLesion dataset. 3D CNNs do not work well with the DeepLesion dataset [87]. The authors of this paper provided several reasons for it. The sensitivity obtained by this model at 16 false-positive rate was 94.72. The models were tested with 4 Tesla K80 GPUs (48 GB) with an Intel(R) Xeon(R) E5-2640 v2 CPU, and the inference time was 4.2 seconds.

3.2.4 Discussion

One-stage 3D object detection and two-stage 3D object detection techniques are computationally expensive as they use 3D CNNs. So, to fulfill the constraints of GPU, the models send the data in the form of patches of 3D data instead of the entire image. The models [93, 87] assert that 3D CNNs do not work well with the DeepLesion dataset. The 3D models are computationally more expensive, and inference time for such models is more than 2D networks. Hence in this project, 3D models were not chosen for analysis. Instead, 2D object detection techniques were chosen for analysis. Zlocha et al. [97] had shown that the RetinaNet model gives better results on the DeepLesion dataset than other models like Faster RCNN and Mask RCNN. RetinaNet performs well in the COCO dataset. It has an mAP higher than Faster RCNN on the COCO dataset. So, this model was chosen for analysis in this project. In several papers like [2, 46, 39, 38] they use YOLO architecture for the object detection in slices of 3D CT scans. Hence YOLOv3 and YOLOv5 were chosen for analysis in this project.

Methodology

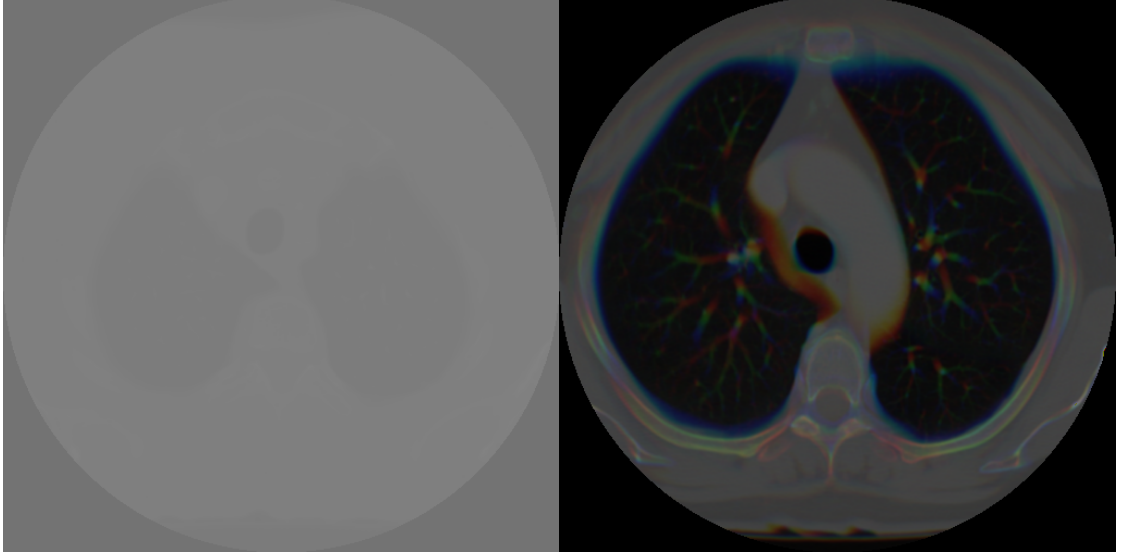
In this chapter, the data processing steps taken in this project are discussed. A brief overview of anchor optimization techniques used in this project is provided. The architecture of three models, i.e., RetinaNet, YOLOv3, and YOLOv5, which are used in this project, are explained.

4.1 Data Processing

Some of the images were marked as possibly noisy. They were removed. The data processing steps are based on [97, 5]. The images are preprocessed as follows, 32768 is subtracted from each pixel intensity to obtain the Hounsfield Unit (HU) values. The HU values are linearly scaled from 0 to 255 and converted into int8 data type. Thus a three-channel image is generated per key slice by concatenating the key slices with a slice 5mm above and below to gain a 3D perspective. The data before preprocessing and after preprocessing is shown in Figure 4.1. As shown in the Table 3.1, some images are not of the resolution 512×512 . Those images were resized to make them of size 512×512 , and the annotations for bounding box coordinates for these images were also scaled accordingly.

4.2 Anchor optimization techniques

In this project, the major work was to test the object detection models at different resolutions. As the resolution of the image is varied, the size of the bounding box also changes accordingly. As the size of the bounding box changes,



(a) Actual Data [89]

(b) Processed Data [89]

Figure 4.1: Image from DeepLesion dataset[89] before and after processing

the anchors generated should be suitable for detecting those bounding boxes. This is achieved by using anchor optimization techniques. YOLOv3 and YOLOv5 utilize K-means clustering for anchor optimization. Hence K-means clustering was applied to set anchor parameters for the RetinaNet model, but the model did not perform well. In the paper [97], they used this differential evolution algorithm[73] for anchor optimization. Hence, this algorithm was used to set anchor parameters for the RetinaNet model in this project.

4.2.1 Differential Evolution algorithm [73]

The explanation for the Differential Evolution algorithm is taken from these sources [80, 73]. Storn et al. [73] introduced this algorithm. This algorithm is stochastic in nature. It does not rely on gradient methods to find the minimum. This algorithm can be used to search large areas of space (candidate space). One of the significant drawbacks of this algorithm is that it requires many function evaluations compared to gradient-based techniques. Hence we used this algorithm to do anchor optimization. This algorithm is provided by scipy library [80] which we used in this project. The function to calculate IoU overlap is taken as the

objective function. The bounding box coordinates are fed as values. The differential evolution algorithm then outputs the prior anchor boxes for training.

4.3 RetinaNet

The explanation of this section is based on [45, 85]. Lin et al. [45] introduced RetinaNet. In the case of R-CNN framework[25], the first stage produces a set of candidate locations, and the second stage classifies the candidate location as either foreground or background using a CNN. The two-stage networks which belong to the R-CNN family[67, 24, 30] performed better than the one-stage networks like YOLO[64, 66] and SSD[21, 49]. Class imbalance is identified as the main obstacle in the case of one-stage detectors by this paper. To eradicate this problem approaches like bootstrapping [74, 68] and hard example mining[78, 18, 71] are used. In this paper, a loss function referred to as focal loss is introduced to tackle the problem of class imbalance.

Focal loss The focal loss is used to address the issue of severe imbalance between background and foreground classes during training. The focal loss is based on the binary cross-entropy(CE) loss.

The binary CE loss is given by equation 4.1:

$$CE(p, y) = \begin{cases} -\log(p) & \text{if } y = 1 \\ -\log(1 - p) & \text{otherwise} \end{cases} \quad (4.1)$$

In equation 4.1 $y \in \{\pm 1\}$ is the ground truth class, $p \in [0, 1]$ is the estimated probability of model with class label $y = 1$. For the convenience of notation p_t is referred to as

$$p_t = \begin{cases} p & \text{if } y = 1 \\ 1 - p & \text{otherwise} \end{cases} \quad (4.2)$$

Therefore,

$$CE(p, y) = CE(p_t) = -\log(p_t) \quad (4.3)$$

$CE(p, y)$ is the blue curve in Figure 3.2.

One of the properties of this loss function is that loss of non-trivial magnitude is incurred by the examples which are easily classified. When summed over a significant number of simple examples, these minute loss values overwhelm the rare

class. A usual way to address this class imbalance is to use a weighting factor for the positive class, i.e., class 1 is $\alpha \in [0, 1]$. Similarly, the weighting factor of the negative class, i.e., -1, is $1 - \alpha$. The CE loss, which is balanced by weighting factor α is given by

$$\text{CE}(p_t) = -\alpha_t \log(p_t) \quad (4.4)$$

This loss is an extension to CE loss which is the building block for focal loss. The severe class imbalance faced during the training of the detection network overwhelms the CE loss. The negatives which are classified constitute the major amount of loss and dominates the gradient. α balances the significance of negative/positive examples. It does not differentiate between hard/easy examples. Thus the authors of this paper propose a loss function that down-weights the examples which are easily classified and emphasize on the training of hard negatives. They include a modulating factor $(1 - p_t)^\gamma$ to the CE loss. This modulating factor has a tunable parameter $\gamma \geq 0$. The focal loss is defined as

$$\text{FL}(p_t) = -(1 - p_t)^\gamma \log(p_t) \quad (4.5)$$

For several values of $\gamma \in [0, 5]$ the focal loss is visualized in Figure 3.2. Focal loss has two properties. The first property is that when the example is wrongly classified, and p_t is not large, the modulating factor α is near one, and the loss function remains unaffected. As p_t tends to one, the modulating factor approaches zero, and the loss for examples which are well classified are down-weighted. When $\gamma = 0$, focal loss is equivalent to CE loss. As γ is increased, the impact of modulating factor also increases. In real-world applications α -balanced focal loss is utilized which is given by:

$$\text{FL}(p_t) = -\alpha_t (1 - p_t)^\gamma \log(p_t) \quad (4.6)$$

RetinaNet network consists of two task-specific subnetworks and a backbone network. The backbone network is responsible for calculating the convolutional feature map over the full input image. The output of the backbone is taken as input by the first subnetwork to perform object classification. The bounding box

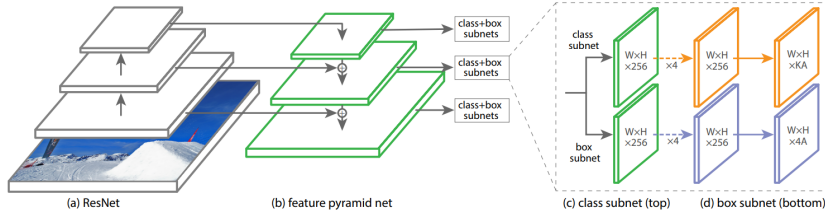


Figure 4.2: Subnetworks of RetinaNet model [45]

regression is given by the second subnetwork. The two subnetworks from the architecture of RetinaNet as shown in Figure 4.2.

4.4 YOLO

In this section the various architectures of YOLO like YOLOv1[66], YOLOv2[64], YOLOv3[65] and YOLOv5[35] are discussed.

4.4.1 You Only Look Once

The explanation of this section is based on [66, 85]. Redmon et al. [66] introduced You Only Look Once(YOLO). In the case of YOLO, the task of object detection is taken as a regression problem. The entire process of detection is accomplished using only one network. To achieve this, the entire image is fragmented into $S \times S$ cells. The center of objects is pivotal in the case of YOLO. If the center lies in a particular cell, that cell is accountable for detecting the object. Three things are predicted by every cell, namely:

- Location of B bounding boxes
- A confidence score
- Class probability

Let $(x1, y1)$ be the coordinates of the top left corner of the bounding box and $(x2, y2)$ be the coordinates of the bottom right corner of the bounding box. The bounding box coordinates are represented by a tuple that consists of 4 values. These values are:

- x-center: It is the x-coordinate of the center of the bounding box
- y-center: It is the y-coordinate of the center of the bounding box
- w: It is the width of the bounding box. Then $w = x2 - x1$
- h: It is the height of the bounding box. Then $h = y2 - y1$

To ensure that the bounding box coordinates, i.e., $(x - center, y - center, w, h)$ lie in the range between zero and one, the bounding box coordinates are normalized by utilizing the width and height of the image. The confidence score represents the likelihood that the cell has an object. Mathematically confidence score is given by:

$$Pr(Object) * IoU_{pred}^{truth} \quad (4.7)$$

In the case where there are no objects in the cell, the confidence score has to be zero. For every cell C class probabilities are generated given by $Pr(Class_i | Object)$. A cell can have many bounding boxes, but only one set of class probabilities are predicted for each cell. At test time, the conditional class probabilities are multiplied with the individual bounding box confidence predictions.

$$Pr(Class_i | Object) * Pr(object) * IoU_{pred}^{truth} = Pr(Class_i) * IoU_{pred}^{truth} \quad (4.8)$$

Every image has $S \times S \times B$ bounding boxes. Every bounding box corresponds to these things, one confidence score, four bounding box coordinates, and C class probabilities. Hence the shape of prediction values for each image is given by $S \times S \times (5B + C)$. This is well illustrated by Figure 4.3.

Architecture of YOLO

The base of the YOLO model is very similar to the GoogLeNet model[75] with changes like the inception module being substituted by 1×1 and 3×3 convolutional layers. The network consists of twenty-four convolutional layers, which are followed by two fully connected layers. The final output has a shape $S \times S \times (5B + C)$. The network architecture is well illustrated in Figure 4.4.

Loss function

The loss function has two parts. The first one is localization loss that provides the bounding box offset prediction. The second one is classification loss that provides the conditional class probabilities. Both of these parts are calculated as the sum-squared error. Classification and localization loss are given equal importance by the sum-squared error loss function. This makes the model unstable as some cells do not have any object, and the gradients of those cells get boosted. To avoid this, localization loss needs to be increased, and classification loss needs to be reduced. This is achieved by using two parameters, the first parameter is λ_{coord} and second parameter is λ_{noobj} . λ_{coord} is set to five and multiplied to bounding box loss and

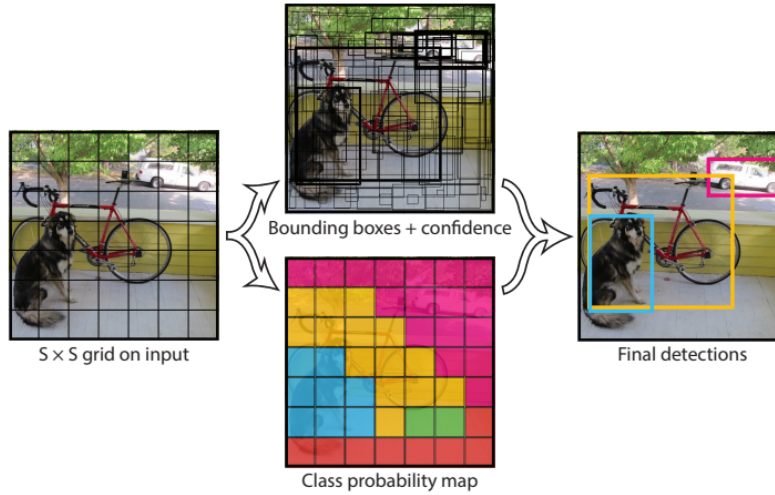


Figure 4.3: Working of YOLO model [66]

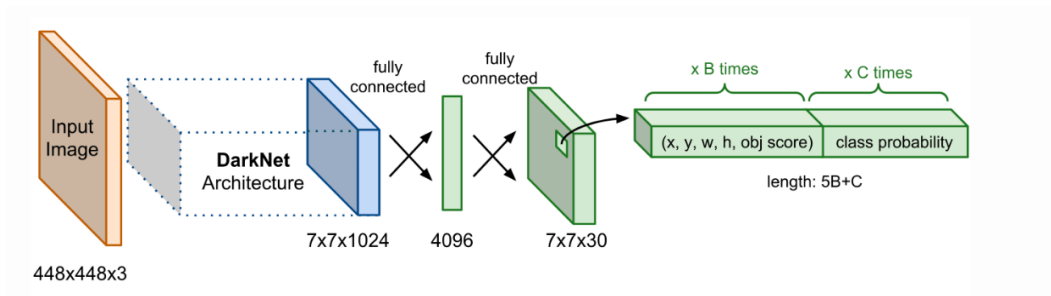


Figure 4.4: YOLO Architecture [85]

λ_{coord} is set to 0.5 and multiplied to classification loss. The loss function for YOLO is given by these equations:

$$p = \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B 1_{ij}^{\text{obj}} [(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2] \quad (4.9)$$

$$q = \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B 1_{ij}^{\text{obj}} \left[\left(\sqrt{w_i} - \sqrt{\hat{w}_i} \right)^2 + \left(\sqrt{h_i} - \sqrt{\hat{h}_i} \right)^2 \right] \quad (4.10)$$

$$\mathcal{L}_{\text{loc}} = p + q \quad (4.11)$$

$$L_{\text{cls}} = \sum_{i=0}^{S^2} \sum_{j=0}^B 1_{ij}^{\text{obj}} (C_i - \hat{C}_i)^2 + \sum_{i=0}^{S^2} 1_i^{\text{obj}} \sum_{c \in \text{classes}} (p_i(c) - \hat{p}_i(c))^2 \quad (4.12)$$

$$\mathcal{L} = \mathcal{L}_{\text{loc}} + \mathcal{L}_{\text{cls}} \quad (4.13)$$

- 1_i^{obj} : A function to ascertain whether object is in cell i.
- 1_{ij}^{obj} : It helps to find if the j -th bounding box of the given cell i is accountable for the prediction of object.
- C_i : Cell i's confidence score
- $\hat{C}_i$: Confidence score predicted by the model
- C : It represents the classes(set of classes)
- $p_i(c)$: The conditional probability that determines if cell i has an object wich belongs to class $c \in \mathcal{C}$.
- $\hat{p}_i(c)$: It represents the predicted class probability.

The classification error is only penalized by the loss function if the object is there in that cell, i.e., $1_i^{\text{obj}} = 1$. The bounding box error is penalized only if the predictor is accountable for the ground truth bounding box, $1_{ij}^{\text{obj}} = 1$ [85].

4.4.2 YOLO9000

The explanation of this section is based on [64, 85]. Redmon et al. [64] introduced YOLO9000, also referred to as YOLOv2. It is an improved version of YOLO. A variety of modifications were applied to YOLO to improve it. Some of them are listed below:

- **Batch Normalization:** Batch normalization leads to significant enhancement in convergence, and adding batch normalization on the convolutional layers in YOLO improves the mAP by two percent.
- **High-Resolution Classifier:** Fine-tuning the model using high-resolution images enhances the detection performance.
- **Convolutional With Anchor Boxes:** Instead of predicting the position of the bounding box utilizing the fully connected layers throughout the entire feature map, convolutional layers are used to predict the positions of anchor boxes, similar to faster R-CNN [67]. The prediction of class probabilities and spatial locations are decoupled. This change reduces the mAP slightly but increases the recall[85].
- **Dimension Clusters:** In contrast to faster R-CNN [67] which utilizes anchor boxes of sizes that are hand-picked ,YOLOv2 uses k-means clustering on the data to get suitable priors on the anchor box sizes. The distance metric relies on the IoU scores:

$$d(\text{ box , centroid }) = 1 - \text{IOU}(\text{box, centroid}) \quad (4.14)$$

Using k-means clustering to generate bounding boxes helps in better representation and the task to learn becomes easier.

- **Direct location prediction:** One of the issues faced by YOLO is the instability of the model during early iterations. This instability could be attributed to the prediction of (x, y) center locations for the bounding box. So to remedy this, location coordinates are generated with respect to the

location of the grid cell. Logistic activation is utilized to ensure that the prediction of location coordinates could be constrained. Five coordinates are predicted for each bounding box. The five coordinates are t_x, t_y, t_w, t_h and t_o . If the size of the anchor box is (p_w, p_h) with respect to the grid cell such that the top left corner is represented by (c_x, c_y) , then the predicted bounding box is given by the equation 4.15 :

$$\begin{aligned}
 b_x &= \sigma(t_x) + c_x \\
 b_y &= \sigma(t_y) + c_y \\
 b_w &= p_w e^{t_w} \\
 b_h &= p_h e^{t_h} \\
 \Pr(\text{object}) * IOU(b, \text{object}) &= \sigma(t_o)
 \end{aligned} \tag{4.15}$$

Hence the prediction of the bounding box is constrained, and the stability of the model is enhanced.

- **Fine-Grained Features:** In YOLOv2, a passthrough layer is added so that the fine-grained features from the beginning layers could be brought to the final output layer. The passthrough layer is designed in such a way that it can extract higher-dimensional features present in the prior layers (like identity mappings present in the ResNet)[85].
- **Multi-scale training:** Input dimension is randomly sampled after every ten batches so that the model becomes robust to input images of various sizes. The input dimension of the image is downsampled by the convolutional layer by a factor of thirty-two. Hence the new dimension is always a multiple of thirty-two.
- **Light-weighted base model:** DarkNet-19 architecture is used as backbone in YOLOv2. DarkNet-19 architecture has nineteen convolutional layers and five max-pooling layers. Hence the base model is light weighted and makes the prediction faster. The main idea is to insert 1x1 convolutional filters and global average pooling between 3x3 convolutional layers. Feature representations are compressed by 1x1 convolutional filters and global average

pooling is used to make predictions.

4.4.3 YOLOv3

The explanation of this section is based on [65, 85]. Redmon et al. [65] introduced YOLOv3, an enhanced version of YOLOv2. The major changes that were made are listed below:

- **Bounding Box Prediction:** In YOLOv3, logistic regression is utilized as loss function to get confidence scores.
- **Class Prediction:** YOLOv3 utilizes multiple logistic classifiers instead of a single softmax layer to predict the class confidence. This is very useful as one image could have several labels, and these labels may not be mutually exclusive.

The feature map from two previous layers is taken, and it is upsampled two times.

- **Predictions Across Scales:** In YOLOv3, after the base feature extraction network, several convolutional layers are added, which makes predictions across three different scales between these convolutional layers. This enables the network to deal with multiple bounding box candidates of several sizes.
- **Skip-layer concatenation:** In YOLOv3, cross-layer connections are added between two prediction layers as well as earlier fine-grained feature maps, except the output layer. The coarse feature maps are upsampled and concatenated with previous features. This concatenation of feature maps helps the network to detect small objects[85].

4.4.4 YOLOv5

YOLOv5 [35] was introduced by the company Ultralytics. In YOLOv5 neck for feature aggregation is Path Aggregation Network (PANet) [48] and the model utilizes Cross Stage Partial Network (CSPNet)[82] as backbone. These improvements have

made the model better for feature extraction, and it has provided the model with a notable boost in the mAP score[15]. YOLOv5 has four different variants, namely YOLOv5s, YOLOv5m, YOLOv5l, and YOLOv5x, respectively. The suffixes s , m , l and x refer to small, medium, large and extra-large. The main difference between these variants is the width and depth of the network, i.e., YOLOv5s has the least depth and width, therefore referred by suffix s , i.e., small followed by YOLOv5m, i.e., medium, and so forth. The network was trained on the COCO dataset. The comparison of results is shown in Figure 4.5.

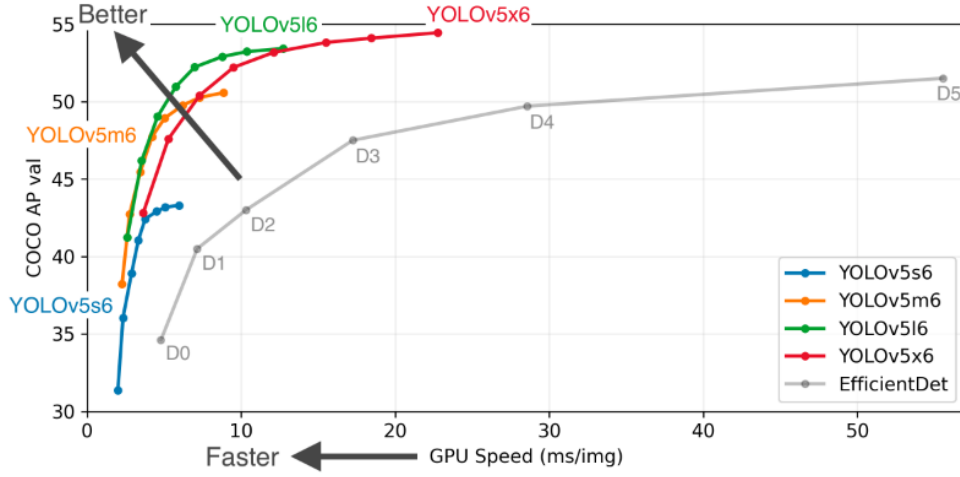


Figure 4.5: Comparison results of YOLOv5 [35]

Evaluation

In this chapter, we discuss the various experiments that were conducted for the analysis of the models. The code for implementation of the RetinaNet model is based on [20]. The code for implementation of YOLOv3 is based on [34]. Similarly, the code for implementation of YOLOv5 is based on [35]. The experiments were conducted on the GPU cluster. The GPU used was Nvidia Tesla V100. The models were trained for 50 epochs. The batch size was set to 8. For evaluation on the test set, the IoU threshold was set to 0.5, and the confidence threshold or score threshold was set to 0.001. The mAP was thus calculated for IoU threshold 0.5. In the case of the RetinaNet model, the anchor parameters were obtained using the Differential Evolution algorithm[73]. All the experiments are conducted on the DeepLesion dataset[89]. The task was to detect lesions and draw bounding boxes around them. Seventy percent of the data was used for training, 15 percent for validation, and 15 percent for testing. There was a substantial variation in the inference time when a shared GPU cluster was used. Hence inference time was measured using 64-bit Intel(R) Core(TM) i5-4210M CPU with 2.60GHz frequency. In first experiment 5.1 the comparison of RetinaNet model with ResNet-50 and ResNet-101 backbone is performed. In the second experiment 5.2 the variants of YOLOv5, namely YOLOv5x, YOLOv5l, YOLOv5m, and YOLOv5s, are compared. In the third experiment 5.3 the impact of initialization of weights on the performance of models is evaluated. In the fourth experiment 5.4 the mAP of models at various resolutions is compared. In the fifth experiment 5.5 the f1 score of models at

various resolutions is evaluated. In the sixth experiment 5.6 the inference time of various models at different resolutions is compared. In the end 5.7 the analysis of the results is carried out to understand the relationship between resolution, mAP, and inference time.

5.1 Comparison of RetinaNet model with ResNet-50 and ResNet-101 backbone

In this experiment, the performance of RetinaNet model was compared with two different backbones, ResNet-50 and ResNet-101. The RetinaNet model uses pre-trained Imagenet [13] weights as default weights for initialization, and hence same initialization weights were used for this experiment. In Figure 5.1 we can see the mAP of the model with ResNet-50 and ResNet-101 backbone. The validation mAP with ResNet-50 backbone is 0.473 and with ResNet-101 backbone is 0.475, RetinaNet with ResNet-101 backbone has slightly better validation mAP than ResNet-50. The test mAP with ResNet-50 backbone is 0.487 and test mAP with ResNet-101 backbone is also 0.487. The mAP score is the same with both the backbones. Hence, the f1 score was calculated for both the backbones to find out which backbone performs better. In Figure 5.2 it can be seen that RetinaNet with ResNet-50 backbone has f1 score of 0.521 and RetinaNet with ResNet-101 backbone has f1 score of 0.518. ResNet-101 is computationally more expensive than ResNet-50. The average of inference times of 500 images was taken into consideration while conducting this experiment. The inference time of ResNet-50 backbone model with the image from DeepLesion dataset with resolution 512x512 on 64-bit Intel(R) Core(TM) i5-4210M CPU with 2.60GHz frequency is 1.351 seconds and with ResNet-101 backbone is 1.877 second. Since ResNet-50 provides comparable performance to ResNet-101 but has a faster inference time. So, RetinaNet with ResNet-50 backbone was chosen for the rest of the experiments.

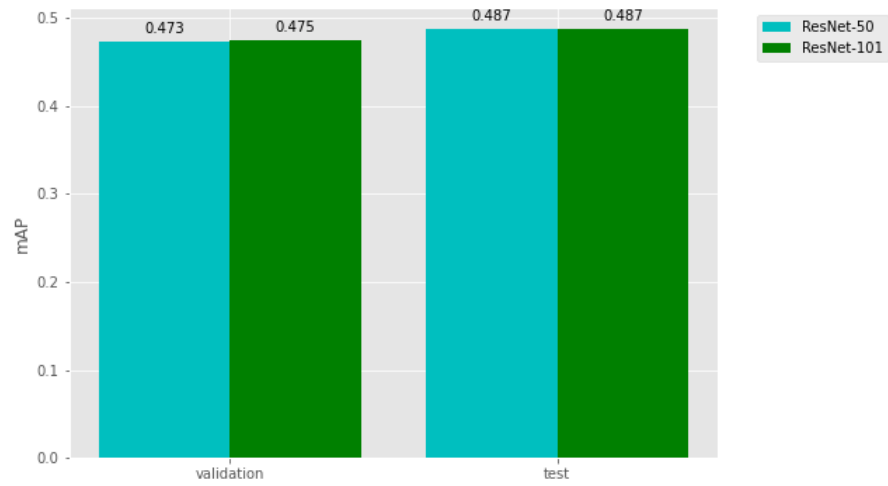


Figure 5.1: mAP plot

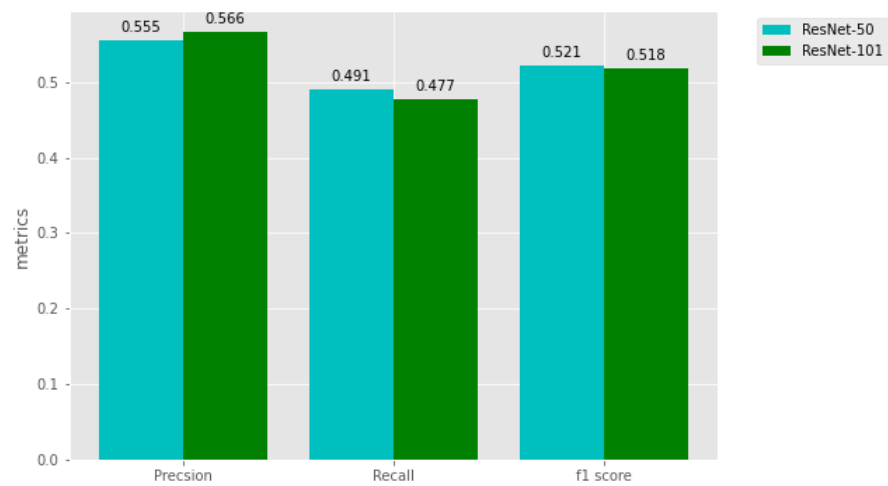


Figure 5.2: Precision, Recall, F1 score plot

5.2 YOLOv5 models

This experiment compares the four different models of YOLOv5, i.e., YOLOv5x, YOLOv5l, YOLOv5m, and YOLOv5s. The main objective behind conducting this experiment is to find out which YOLOv5 model performs the best on the DeepLesion dataset. The experiments were conducted without initializing any weights. In Figure 5.3 it is seen that the YOLOv5x has the highest test as well as validation mAP. Similarly, in Figure 5.4 it is seen that YOLOv5x has the highest f1 score among all four models. Hence YOLOv5x was chosen for carrying out the rest of the experiments.

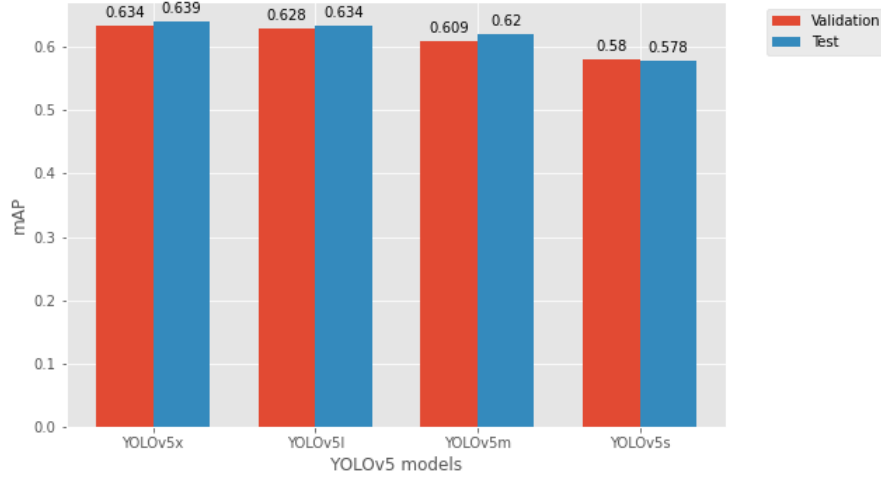


Figure 5.3: YOLOv5 models plot for mAP

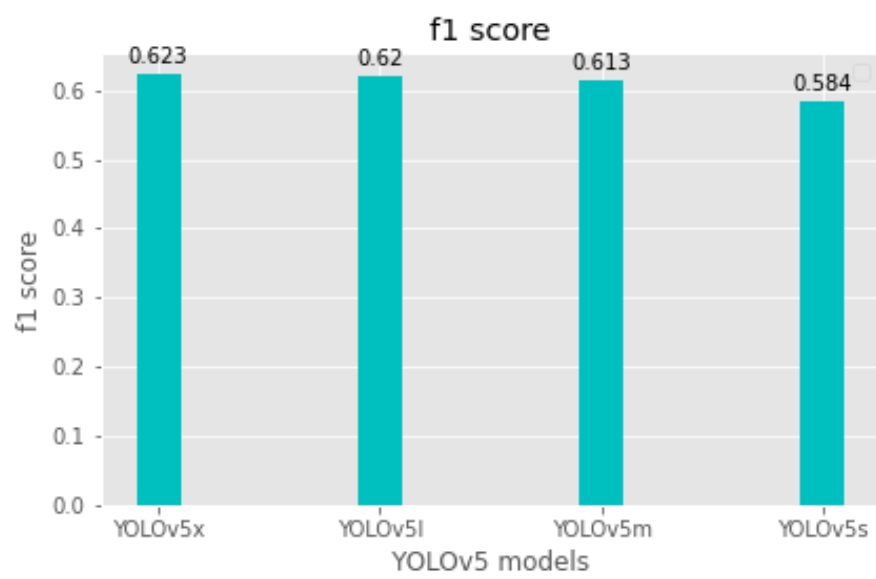


Figure 5.4: YOLOv5 models plot for f1 score

5.3 Initialization with random and pretrained weights

In this experiment, the models were first initialized with random weights and were trained on the DeepLesion dataset. Then the model was initialized with pre-trained COCO [43] weights and trained on the DeepLesion dataset. In Figure 5.5 the comparison of mAP of models can be seen. It is seen that mAP of models improves significantly when the model is initialized with pre-trained COCO weights. It can also be seen that with random initialization of weights and with pre-trained COCO weights, YOLOv5x has the best performance among the three models. On the other hand, with random initialization of weights and pre-trained COCO weights, the mAP of RetinaNet is the least among the three models. In Figure 5.6 the f1 score of the models was compared when the model is initialized with random weights and with pre-trained COCO weights. The results obtained were similar to the one seen in Figure 5.5. F1 score improved significantly when the model was initialized with pre-trained COCO weights instead of initializing it with random weights. Hence for the rest of the experiments, the models were initialized with pre-trained COCO weights.

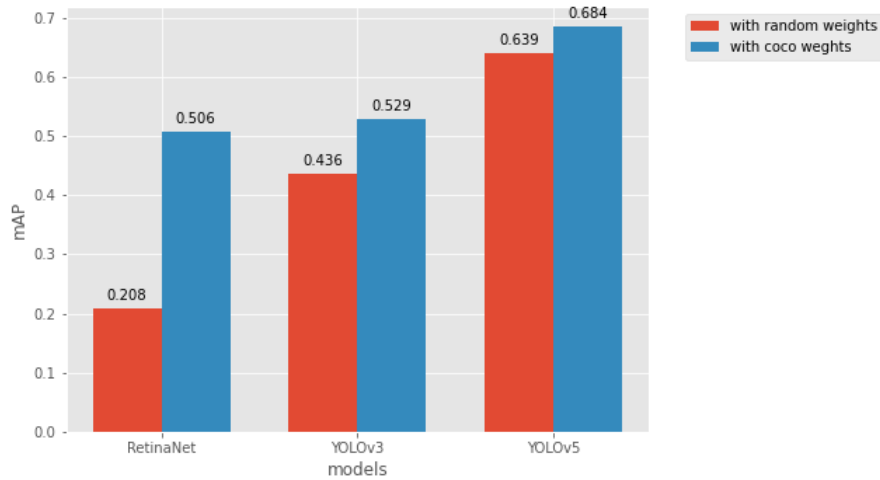


Figure 5.5: Comparison of mAP models with and without initialization weights

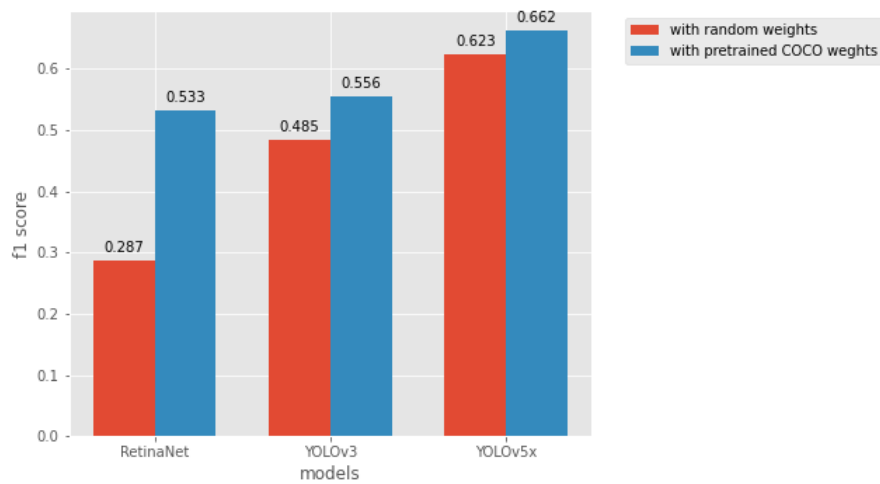


Figure 5.6: Comparison of f1 score of models with and without initialization weights

5.4 Comparison of mAP of models

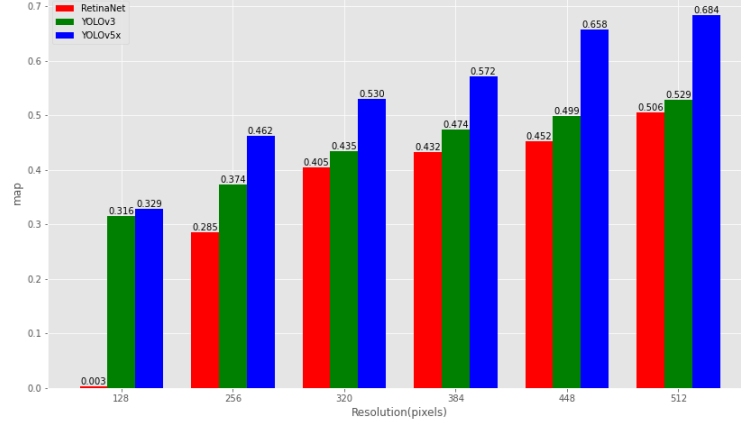


Figure 5.7: Comparison of mAP of models at different resolutions

In this experiment, the mAP of the various models is compared. The objective behind conducting this experiment was to find the model that has the best mAP score even at low resolution. The experiment also was conducted to analyze the impact of resolution on the mAP of the models. The image size must be divisible by two at least six times to avoid fractions when downscaling and upscaling. Hence the resolutions were chosen in such a way that they are the multiple of 32. Training models require a substantial amount of time. So, considering time constraints, only six resolutions were chosen for experiments. In Figure 5.7, the label 512 along the x-axis represents that the image is 512 pixels wide and 512 pixels high. This naming convention is followed in all the experiments related to resolution in this report. As the resolution is varied, the size of the bounding box also changes. Hence, the bounding box coordinates were scaled according to the change in resolution of the image. The resolution of images was changed using the bilinear interpolation technique. In Figure 5.7 it can be seen that YOLOv5x has the best mAP at various resolutions. It can also be observed that YOLOv3 and YOLOv5x outperform the RetinaNet model. The RetinaNet model has an mAP of 0.003 at the resolution 128×128 . The possible explanation behind this behavior could be that the RetinaNet model uses a feature pyramid network, and the features are

downsampled at different levels. Since the resolution is already low, so when it is downsampled, the model does not learn.

5.5 Comparison of f1 score of models

In this experiment, the f1 score of the models is compared at different resolutions. The experiment was conducted to analyze the impact of resolution on the f1 score of the models. In the figure 5.8 it is seen that YOLOv5x has the best f1 score and RetinaNet has the least f1 score when the resolution is varied. The results are similar to what was seen in the experiment 5.4 where the mAP of various models are compared at multiple resolutions. The conclusion that can be drawn from these experiments is that YOLOv5x has the best f1 score at all the different resolutions chosen for conducting the experiments. It is also observed that as the resolution is reduced, the f1 score of the model declines.

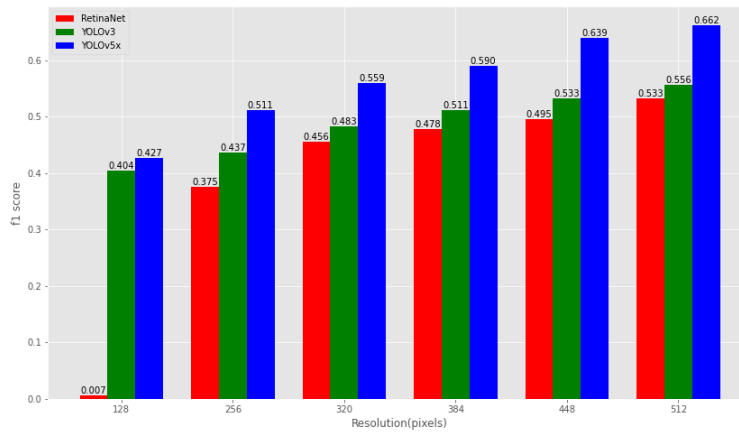


Figure 5.8: Comparison of f1 score of models at different resolutions

5.6 Comparison of inference time of models

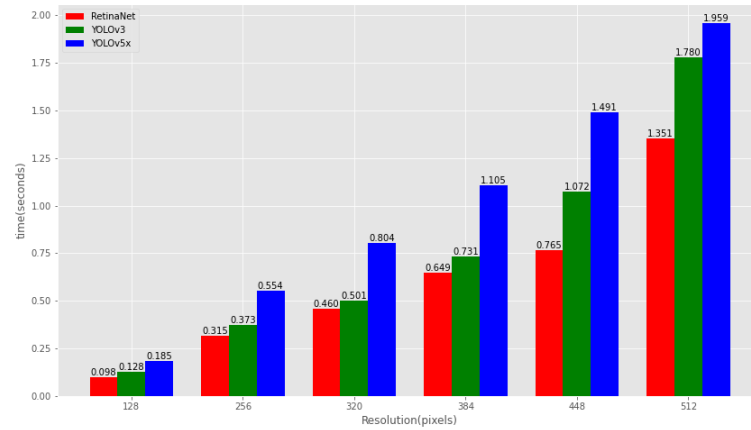


Figure 5.9: Comparison of inference time of models at different resolutions

In this experiment, the inference time of models was compared at various resolutions. The average of inference times of 500 images is taken into consideration while conducting this experiment. The main idea behind conducting this experiment was to determine how the inference time of models varies when the resolution is changed. In Figure 5.9 it can be seen that the YOLOv5x model has the highest inference time, and RetinaNet has the lowest. It can also be observed that as the resolution is reduced, there is a significant drop in inference time.

5.7 Analysis of impact of resolution on inference time and mAP

Resolution	mAP	Inference time	% change in Resolution	% change in in mAP	% change in inference time
512	0.684	1.959	0	0	0
448	0.658	1.491	23.4375	03.80	23.9
384	0.5782	1.105	43.75	15.5	43.6
320	0.530	0.804	60.9375	23.4	59.0
256	0.462	0.554	75.0	33.7	71.7
128	0.329	0.185	93.75	54.0	90.6

Table 5.1: Impact of Resolution on inference time and mAP

In Table 5.1 the values of resolution and the corresponding mAP and inference time for the YOLOv5x model is provided. Along with these values, the change in percentage of the resolution, mAP, and inference time is also provided. It can be seen that when resolution is reduced by a certain percentage, the percentage change in mAP is less compared to the percentage change in inference. Hence it could be asserted that sacrificing mAP for improving the inference time by reducing resolution is feasible. Out of these various resolutions, the resolution 448×448 has better mAP and inference time, so the resolution 448×448 could be chosen for real-world applications, where speed and mAP both are of importance.

Conclusion and Future work

6.1 Conclusion

In this work, our main focus was to assess the impact of resolution on the mAP, F1 score, and inference time of the models. To assess this, a number of experiments were conducted. The major conclusions that could be drawn from these experiments are listed below:

- YOLOv5x has the best mAP and f1 score compared to RetinaNet and YOLOv3 at various resolutions on the DeepLesion dataset.
- RetinaNet model has the least inference time compared to YOLOv5x and YOLOv3 at various resolutions on the DeepLesion dataset.
- From the experiments, it could be asserted that sacrificing mAP for improving the inference time by reducing resolution is feasible.

6.2 Lessons learned

In this work, the lessons that were learned are as follows:

- As the resolution is varied, the size of the bounding box changes, and similarly, the anchor parameters also need to be changed. So, anchor optimization is of key importance. One object detection which performs well with one sort of anchor optimization technique may not perform well with other kind of

anchor optimization techniques. So, selecting a suitable anchor optimization technique for a particular model is necessary.

- There was a substantial variation in the inference time when a shared GPU cluster was used. Hence shared GPU cluster should not be used for measuring inference time.

6.3 Future work

This work has made us scrutinize over the possible directions in which the work could proceed. Some of them are listed below:

- In this work, we concatenated three slices of the image during data processing. However, volume data has many slices; we could try to concatenate more slices and develop a model that could accept data from multiple(more than 3) slices and analyze how using multiple channels could impact the computational complexity, speed, and accuracy of the model.
- In this work, due to time limitation, only one type of technique to vary the resolution, i.e., bilinear interpolation technique, was used. But there are several other techniques like nearest-neighbor interpolation, bicubic interpolation, Lanczos interpolation, etc. And one of the possible future work would be to prepare datasets with each of these types of approaches to change the resolution and analyze the performance of models.
- The entire work is based on the idea that resolution could be reduced to make object detection systems faster. But there could be other techniques that could be used to make object detection systems faster. This is one of the things that is worth exploring.

Bibliography

- [1] Fetulhak Abdurahman. <https://tinyurl.com/2w562ma7>. Accessed on 2 April 2021.
- [2] Saeedeh Afshari, Aïcha BenTaieb, and Ghassan Hamarneh. Automatic localization of normal active organs in 3D PET scans. *Computerized Medical Imaging and Graphics*, 70:111–118, 2018.
- [3] Bogdan Alexe, Thomas Deselaers, and Vittorio Ferrari. Measuring the objectness of image windows. *IEEE transactions on pattern analysis and machine intelligence*, 34(11):2189–2202, 2012.
- [4] LUNA16 Lung Nodule Analysis. <https://luna16.grand-challenge.org/>. Accessed on 22 October 2020.
- [5] Aniruddha and Parv Saxena. <https://github.com/anir16293/Deep-Lesion>. Accessed on 19 August 2021.
- [6] General Blockchain. <https://github.com/GeneralBlockchain/covid-19-chest-xray-lung-bounding-boxes-dataset>. Accessed on 22 October 2020.
- [7] Joao Carreira and Cristian Sminchisescu. CPMC: Automatic object segmentation using constrained parametric min-cuts. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 34(7):1312–1328, 2011.
- [8] Hao Chen, Qi Dou, Lequan Yu, and Pheng-Ann Heng. VoxResNet: Deep voxelwise residual networks for volumetric brain segmentation. *arXiv preprint arXiv:1608.05895*, 2016.

-
- [9] Nello Cristianini and Elisa Ricci. *Support Vector Machines*, pages 928–932. Springer US, Boston, MA, 2008. ISBN 978-0-387-30162-4. doi: 10.1007/978-0-387-30162-4_415. URL https://doi.org/10.1007/978-0-387-30162-4_415.
 - [10] Jifeng Dai, Yi Li, Kaiming He, and Jian Sun. R-fcn: Object detection via region-based fully convolutional networks. In *Advances in neural information processing systems*, pages 379–387, 2016.
 - [11] Navneet Dalal and Bill Triggs. Histograms of oriented gradients for human detection. In *2005 IEEE computer society conference on computer vision and pattern recognition (CVPR’05)*, volume 1, pages 886–893. Ieee, 2005.
 - [12] DeepAI. <https://deepai.org/machine-learning-glossary-and-terms/stride>. Accessed on 22 October 2020.
 - [13] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition*, pages 248–255. Ieee, 2009.
 - [14] Jia Ding, Aoxue Li, Zhiqiang Hu, and Liwei Wang. Accurate pulmonary nodule detection in computed tomography images using deep convolutional neural networks. In *International Conference on Medical Image Computing and Computer-Assisted Intervention*, pages 559–567. Springer, 2017.
 - [15] Yuchen Ding, Zichen Li, and David Yastremsky. Real-time face mask detection in video data. *arXiv preprint arXiv:2105.01816*, 2021.
 - [16] Martin Engelcke, Dushyant Rao, Dominic Zeng Wang, Chi Hay Tong, and Ingmar Posner. Vote3deep: Fast object detection in 3d point clouds using efficient convolutional neural networks. In *2017 IEEE International Conference on Robotics and Automation (ICRA)*, pages 1355–1361, 2017. doi: 10.1109/ICRA.2017.7989161.
 - [17] Mark Everingham, Luc Van Gool, Christopher K. I. Williams, John Winn, and Andrew Zisserman. The pascal visual object classes (voc) challenge. In *International Journal of Computer Vision*, 2014.

- [18] Pedro F Felzenszwalb, Ross B Girshick, and David McAllester. Cascade object detection with deformable part models. In *2010 IEEE Computer society conference on computer vision and pattern recognition*, pages 2241–2248. Ieee, 2010.
- [19] Di Feng, Xiao Wei, Lars Rosenbaum, Atsuto Maki, and Klaus Dietmayer. Deep active learning for efficient training of a lidar 3d object detector. In *2019 IEEE Intelligent Vehicles Symposium (IV)*, pages 667–674. IEEE, 2019.
- [20] Fizyr. https://github.com/fizyr/keras-retinanet/tree/master/keras_retinanet. Accessed on 22 October 2020.
- [21] Cheng-Yang Fu, Wei Liu, Ananth Ranga, Ambrish Tyagi, and Alexander C Berg. Dssd: Deconvolutional single shot detector. *arXiv preprint arXiv:1701.06659*, 2017.
- [22] Hao Gao. <https://medium.com/@smallfishbigsea/understand-ssd-and-implement-your-own-caa3232cd6ad>. Accessed on 06 June 2021.
- [23] GeeksforGeeks. <https://www.geeksforgeeks.org/yolo-you-only-look-once-real-time-object-detection/>. Accessed on 06 June 2021.
- [24] Ross Girshick. Fast r-cnn. In *Proceedings of the IEEE international conference on computer vision*, pages 1440–1448, 2015.
- [25] Ross Girshick, Jeff Donahue, Trevor Darrell, and Jitendra Malik. Rich feature hierarchies for accurate object detection and semantic segmentation. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 580–587, 2014.
- [26] Ross Girshick, Jeff Donahue, Trevor Darrell, and Jitendra Malik. Region-based convolutional networks for accurate object detection and segmentation. *IEEE transactions on pattern analysis and machine intelligence*, 38(1):142–158, 2015.

-
- [27] Ian Goodfellow, David Warde-Farley, Mehdi Mirza, Aaron Courville, and Yoshua Bengio. Maxout networks. In *International conference on machine learning*, pages 1319–1327. PMLR, 2013.
- [28] Yulan Guo, Hanyun Wang, Qingyong Hu, Hao Liu, Li Liu, and Mohammed Bennamoun. Deep learning for 3d point clouds: A survey. *IEEE transactions on pattern analysis and machine intelligence*, 2020.
- [29] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2016.
- [30] Kaiming He, Georgia Gkioxari, Piotr Dollár, and Ross Girshick. Mask r-cnn. In *Proceedings of the IEEE international conference on computer vision*, pages 2961–2969, 2017.
- [31] Gao Huang, Zhuang Liu, Laurens Van Der Maaten, and Kilian Q Weinberger. Densely connected convolutional networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 4700–4708, 2017.
- [32] iData Research. <https://tinyurl.com/2s85xjx5>. Accessed on 22 October 2020.
- [33] Licheng Jiao, Fan Zhang, Fang Liu, Shuyuan Yang, Lingling Li, Zhixi Feng, and Rong Qu. A survey of deep learning-based object detection. *IEEE access*, 7:128837–128868, 2019.
- [34] Glenn Jocher, Yonghye Kwon, guigarfr, perry0418, Josh Veitch-Michaelis, Ttayyu, Daniel Suess, Fatih Baltac, Gabriel Bianconi, IlyaOvodov, Marc, e96031413, Chang Lee, Dustin Kendall, Falak, Francisco Reveriano, FuLin, GoogleWiki, Jason Nataprawira, Jeremy Hu, LinCoce, LukeAI, NanoCode012, NirZarrabi, Oulbacha Reda, Piotr Skalski, SergioSanchezMontesUAM, Shiwei Song, Thomas Havlik, and Timothy M. Shead. ultralytics/yolov3: v9.5.0 - YOLOv5 v5.0 release compatibility update for YOLOv3, April 2021. URL <https://doi.org/10.5281/zenodo.4681234>.

- [35] Glenn Jocher, Alex Stoken, Jirka Borovec, NanoCode012, Ayush Chaurasia, TaoXie, Liu Changyu, Abhiram V, Laughing, tkianai, yxNONG, Adam Hogan, lorenzomamma, AlexWang1900, Jan Hajek, Laurentiu Diaconu, Marc, Yonghye Kwon, oleg, wanghaoyang0106, Yann Defretin, Aditya Lohia, ml5ah, Ben Milanko, Benjamin Fineran, Daniel Khromov, Ding Yiwei, Doug, Durgesh, and Francisco Ingham. ultralytics/yolov5: v5.0 - YOLOv5-P6 1280 models, AWS, Supervise.ly and YouTube integrations, April 2021. URL <https://doi.org/10.5281/zenodo.4679653>.
- [36] Evi Kopelowitz and Guy Engelhard. Lung nodules detection and segmentation using 3d mask-rcnn. *arXiv preprint arXiv:1907.07676*, 2019.
- [37] Larxel. <https://www.kaggle.com/andrewmvd/covid19-ct-scans>. Accessed on 22 October 2020.
- [38] Cristina Oyarzun Laura, Patrick Hofmann, Klaus Drechsler, and Stefan Wesarg. Automatic detection of the nasal cavities and paranasal sinuses using deep neural networks. In *In IEEE 16th International Symposium on Biomedical Imaging, pages 1154-1157. IEEE*, 2019.
- [39] Andréanne Lemay. Kidney recognition in ct using yolov3. *arXiv preprint arXiv:1910.01268*, 2019.
- [40] Bo Li. 3D fully convolutional network for vehicle detection in point cloud. In *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 1513–1518. IEEE, 2017.
- [41] Fangzhou Liao, Ming Liang, Zhe Li, Xiaolin Hu, and Sen Song. Evaluate the malignancy of pulmonary nodules using the 3-d deep leaky noisy-or network. *IEEE transactions on neural networks and learning systems*, 30(11):3484–3495, 2019.
- [42] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C Lawrence Zitnick. Microsoft coco: Common objects in context. In *European conference on computer vision*, pages 740–755. Springer, 2014.

-
- [43] Tsung-Yi Lin, Michael Maire, Serge J. Belongie, Lubomir D. Bourdev, Ross B. Girshick, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollr, and C. Lawrence Zitnick. Microsoft coco: Common objects in context. In *arXiv:1405.0312*, 2015.
- [44] Tsung-Yi Lin, Piotr Dollár, Ross Girshick, Kaiming He, Bharath Hariharan, and Serge Belongie. Feature pyramid networks for object detection. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 2117–2125, 2017.
- [45] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal loss for dense object detection. In *Proceedings of the IEEE international conference on computer vision*, pages 2980–2988, 2017.
- [46] Chenyang Liu, Shen-Chiang Hu, Chunhao Wang, Kyle Lafata, and Fang-Fang Yin. Automatic detection of pulmonary nodules on ct images with yolov3: development and evaluation using simulated and patient data. *Quantitative Imaging in Medicine and Surgery*, 10(10):1917, 2020.
- [47] Li Liu, Wanli Ouyang, Xiaogang Wang, Paul Fieguth, Jie Chen, Xinwang Liu, and Matti Pietikäinen. Deep learning for generic object detection: A survey. *International journal of computer vision*, 128(2):261–318, 2020.
- [48] Shu Liu, Lu Qi, Haifang Qin, Jianping Shi, and Jiaya Jia. Path aggregation network for instance segmentation. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 8759–8768, 2018.
- [49] Wei Liu, Dragomir Anguelov, Dumitru Erhan, Christian Szegedy, Scott Reed, Cheng-Yang Fu, and Alexander C Berg. Ssd: Single shot multibox detector. In *European conference on computer vision*, pages 21–37. Springer, 2016.
- [50] Weizeng Lu, Xi Jia, Wei Chen, Nicolo Savioli, Antonio de Marvao, Linlin Shen, Declan O’Regan, and Jinming Duan. One-stage multi-task detector for 3d cardiac mr imaging. In *2020 25th International Conference on Pattern Recognition (ICPR)*, pages 1949–1955. IEEE, 2021.

- [51] Jerubbaal John Luke, Rajkumar Joseph, and Mahesh Balaji. Impact of image size on accuracy and generalization of convolutional neural networks. In *IJRAR*, 2019.
- [52] Wenshuo Ma, Tingzhong Tian, Hang Xu, Yimin Huang, and Zhenguo Li. Aabo: Adaptive anchor box optimization for object detection via bayesian sub-sampling. In *European Conference on Computer Vision*, pages 560–575. Springer, 2020.
- [53] Bjoern H Menze, Andras Jakab, Stefan Bauer, Jayashree Kalpathy-Cramer, Keyvan Farahani, Justin Kirby, Yuliya Burren, Nicole Porz, Johannes Slotboom, Roland Wiest, et al. The multimodal brain tumor image segmentation benchmark (brats). *IEEE transactions on medical imaging*, 34(10):1993–2024, 2014.
- [54] microscope.org. <https://microscope-microscope.org/microscope-info/image-resolution/>. Accessed on 22 October 2020.
- [55] NIH. <https://www.nibib.nih.gov/science-education/science-topics/computed-tomography-ct>, . Accessed on 22 October 2020.
- [56] NIH. <https://www.nibib.nih.gov/science-education/science-topics/magnetic-resonance-imaging-mri>, . Accessed on 22 October 2020.
- [57] Radiological Society of North America. <https://www.rsna.org/education/ai-resources-and-training/ai-image-challenge/rsna-pneumonia-detection-challenge-2018>. Accessed on 22 October 2020.
- [58] Rafael Padilla, Sergio L Netto, and Eduardo AB da Silva. A survey on performance metrics for object-detection algorithms. In *2020 International Conference on Systems, Signals and Image Processing (IWSSIP)*, pages 237–242. IEEE, 2020.
- [59] J. Pearl. *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*. Morgan Kaufmann, 2014.

- [60] Xiaojiang Peng and Cordelia Schmid. Multi-region two-stream r-cnn for action detection. In *European conference on computer vision*, pages 744–759. Springer, 2016.
- [61] Bernhard Preim and Charl Botha. Acquisition of medical image data. *Visual Computing for Medicine*, abs/1804.06882, 2014.
- [62] pyimagesearch. <https://www.pyimagesearch.com/2016/11/07/intersection-over-union-iou-for-object-detection/>. Accessed on 28 April 2021.
- [63] AZO quantum. <https://www.azoquantum.com/News.aspx?newsID=7018>. Accessed on 22 October 2020.
- [64] Joseph Redmon and Ali Farhadi. Yolo9000: better, faster, stronger. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 7263–7271, 2017.
- [65] Joseph Redmon and Ali Farhadi. Yolo3: An incremental improvement. *arXiv preprint arXiv:1804.02767*, 2018.
- [66] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 779–788, 2016.
- [67] Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. Faster r-cnn: Towards real-time object detection with region proposal networks. *Advances in neural information processing systems*, 28:91–99, 2015.
- [68] Henry A Rowley, Shumeet Baluja, Takeo Kanade, et al. *Human face detection in visual scenes*. Citeseer, 1995.
- [69] J Anthony Seibert. Tradeoffs between image quality and dose. *Pediatric radiology*, 34(3):S183–S195, 2004.
- [70] Connor Shorten and Taghi M Khoshgoftaar. A survey on image data augmentation for deep learning. *Journal of Big Data*, 6(1):1–48, 2019.

- [71] Abhinav Shrivastava, Abhinav Gupta, and Ross Girshick. Training region-based object detectors with online hard example mining. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 761–769, 2016.
- [72] Shuran Song, Samuel P Lichtenberg, and Jianxiong Xiao. Sun rgb-d: A rgb-d scene understanding benchmark suite. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 567–576, 2015.
- [73] Rainer Storn and Kenneth Price. Differential evolution—a simple and efficient heuristic for global optimization over continuous spaces. *Journal of global optimization*, 11(4):341–359, 1997.
- [74] Kah-Kay Sung. Learning and example selection for object and pattern detection. 1996.
- [75] Christian Szegedy, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott Reed, Dragomir Anguelov, Dumitru Erhan, Vincent Vanhoucke, and Andrew Rabinovich. Going deeper with convolutions. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 1–9, 2015.
- [76] You-Bao Tang, Ke Yan, Yu-Xing Tang, Jiamin Liu, Jin Xiao, and Ronald M Summers. ULDor: a universal lesion detector for CT scans with pseudo masks and hard negative example mining. In *2019 IEEE 16th International Symposium on Biomedical Imaging (ISBI 2019)*, pages 833–836. IEEE, 2019.
- [77] Jasper RR Uijlings, Koen EA Van De Sande, Theo Gevers, and Arnold WM Smeulders. Selective search for object recognition. *International journal of computer vision*, 104(2):154–171, 2013.
- [78] Paul Viola and Michael Jones. Rapid object detection using a boosted cascade of simple features. In *Proceedings of the 2001 IEEE computer society conference on computer vision and pattern recognition. CVPR 2001*, volume 1, pages I–I. Ieee, 2001.
- [79] Paul Viola and Michael Jones. Rapid object detection using a boosted cascade of simple features. In *Proceedings of the 2001 IEEE computer society conference*

- on computer vision and pattern recognition. CVPR 2001*, volume 1, pages I–I. Ieee, 2001.
- [80] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, Stéfan J. van der Walt, Matthew Brett, Joshua Wilson, K. Jarrod Millman, Nikolay Mayorov, Andrew R. J. Nelson, Eric Jones, Robert Kern, Eric Larson, C J Carey, İlhan Polat, Yu Feng, Eric W. Moore, Jake VanderPlas, Denis Laxalde, Josef Perktold, Robert Cimrman, Ian Henriksen, E. A. Quintero, Charles R. Harris, Anne M. Archibald, Antônio H. Ribeiro, Fabian Pedregosa, Paul van Mulbregt, and SciPy 1.0 Contributors. SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods*, 17:261–272, 2020. doi: 10.1038/s41592-019-0686-2.
- [81] Walber. <https://commons.wikimedia.org/wiki/File:Precisionrecall.svg>. Accessed on 3 September 2021.
- [82] Chien-Yao Wang, Hong-Yuan Mark Liao, Yueh-Hua Wu, Ping-Yang Chen, Jun-Wei Hsieh, and I-Hau Yeh. CSPNet: A new backbone that can enhance learning capability of CNN. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition workshops*, pages 390–391, 2020.
- [83] Robert J Wang, Xiang Li, and Charles X Ling. Pelee: A real-time object detection system on mobile devices. *arXiv preprint arXiv:1804.06882*, 2018.
- [84] Xiaoyu Wang, Ming Yang, Shenghuo Zhu, and Yuanqing Lin. Regionlets for generic object detection. In *Proceedings of the IEEE international conference on computer vision*, pages 17–24, 2013.
- [85] Lilian Weng. Object detection part 4: Fast detection models. *lilianweng.github.io/lil-log*, 2018. URL <http://lilianweng.github.io/lil-log/2018/12/27/object-detection-part-4.html>.
- [86] Saining Xie, Ross Girshick, Piotr Dollár, Zhuowen Tu, and Kaiming He. Aggregated residual transformations for deep neural networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 1492–1500, 2017.

- [87] Ke Yan, Mohammadhadi Bagheri, and Ronald M Summers. 3d context enhanced region-based convolutional neural network for end-to-end lesion detection. In *International Conference on Medical Image Computing and Computer-Assisted Intervention*, pages 511–519. Springer, 2018.
- [88] Ke Yan, Xiaosong Wang, Le Lu, and Ronald M Summers. Deeplesion: automated mining of large-scale lesion annotations and universal lesion detection with deep learning. *Journal of medical imaging*, 5(3):036501, 2018.
- [89] Ke Yan, Xiaosong Wang, Le Lu, Ling Zhang, Adam P Harrison, Mohammadhadi Bagheri, and Ronald M Summers. Deep lesion graphs in the wild: relationship learning and organization of significant radiology image findings in a diverse large-scale lesion database. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 9261–9270, 2018.
- [90] Anqi Yang. *3D Object Detection from CT Scans using a Slice-and-fuse Approach*. PhD thesis, Doctoral dissertation, Robotics Institute, 2019.
- [91] Zetong Yang, Yanan Sun, Shu Liu, Xiaoyong Shen, and Jiaya Jia. Std: Sparse-to-dense 3d object detector for point cloud. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 1951–1960, 2019.
- [92] Syed Sahil Abbas Zaidi, Mohammad Samar Ansari, Asra Aslam, Nadia Kanwal, Mamoon Asghar, and Brian Lee. A survey of modern deep learning based object detection models. *arXiv preprint arXiv:2104.11892*, 2021.
- [93] Ning Zhang, Yu Cao, Benyuan Liu, and Yan Luo. 3d aggregated faster r-cnn for general lesion detection. *arXiv preprint arXiv:2001.11071*, 2020.
- [94] B. Zhao, J. Soraghan, G. Di-caterina, L. Petropoulakis, D. Grose, and T. Doshi. Automatic 3d segmentation of mri data for detection of head and neck cancerous lymph nodes. In *2018 Signal Processing: Algorithms, Architectures, Arrangements, and Applications (SPA)*, pages 298–303, 2018. doi: 10.23919/SPA.2018.8563420.

- [95] Yin Zhou and Oncel Tuzel. Voxelnet: End-to-end learning for point cloud based 3d object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2018.
- [96] Wentao Zhu, Chaochun Liu, Wei Fan, and Xiaohui Xie. Deeplung: Deep 3D dual path nets for automated pulmonary nodule detection and classification. In *2018 IEEE Winter Conference on Applications of Computer Vision (WACV)*, pages 673–681. IEEE, 2018.
- [97] Martin Zlocha, Qi Dou, and Ben Glocker. Improving RetinaNet for CT lesion detection with dense masks from weak RECIST labels. In *International Conference on Medical Image Computing and Computer-Assisted Intervention*, pages 402–410. Springer, 2019.